



UNIVERSIDAD
DE GRANADA

GENERACIÓN DE DATOS SINTÉTICA A PARTIR DE UN
SISTEMA IOT DE DETECCIÓN DE VEHÍCULOS UTILIZANDO
MODELOS TRANSFORMER CON ATENCIÓN EN PYTHON

RICARDO RUIZ FERNÁNDEZ DE ALBA

Trabajo Fin de Grado

Doble Grado en Ingeniería Informática y Matemáticas

Tutores

María Bermúdez Edo

Daniel Bolaños

FACULTAD DE CIENCIAS

E.T.S. INGENIERÍAS INFORMÁTICA Y DE TELECOMUNICACIÓN

Granada, a 27 de abril de 2025

ÍNDICE GENERAL

1. INTRODUCCIÓN Y OBJETIVOS	5
1.1. Introducción	5
1.1.1. Contextualización	5
1.1.2. Descripción del problema	5
1.1.3. Estructura del trabajo	6
1.1.4. Herramientas Matemáticas e Informáticas	6
1.1.5. Bibliografía Fundamental	7
1.2. Objetivos	7
1.2.1. Objetivo General	7
1.2.2. Parte de Matemáticas Específicos	7
1.2.3. Parte de Informática	7
2. FUNDAMENTOS MATEMÁTICOS DEL APRENDIZAJE AUTOMÁTICO	8
3. DEL TRANSFORMER A SAITS: ARQUITECTURA Y APLICACIÓN	9
3.0.1. Introducción a los Modelos Transformer	9
3.0.2. Mecanismo de Atención Escalonada (Scaled Dot-Product At- tention)	9
3.0.3. Estructura de Capas en un Transformer	12
3.0.4. Retropropagación en Modelos Transformer	15
3.1. Self-Attention-based Imputation for Time Series (SAITS)	17
3.1.1. Arquitectura del Modelo SAITS	17
3.1.2. Función de Pérdida para SAITS	18
3.1.3. Ventajas de Self-Attention para Imputación de Series Temporales	19
4. ANÁLISIS EXPLORATORIO DE LOS DATOS Y PIPELINE DE PREPROCESA- MIENTO E IMPUTACIÓN	21
4.1. Preprocesamiento e Imputación	22
4.1.1. Imputación por Mediana	22
4.1.2. Imputación con SAITS	22
5. CONCLUSIONES Y TRABAJO FUTURO	23
5.1. Conclusiones	23
5.1.1. Principales Contribuciones del Trabajo	23
5.1.2. Impacto del Uso de Datos Sintéticos en IoT	23
5.2. Trabajo Futuro	23
5.2.1. Mejora de los Modelos Transformer	23
5.2.2. Extensiones a Otros Sistemas IoT	23
5.2.3. Posibles Aplicaciones en Diferentes Dominios	23
Anexos	24
Anexo A: Cronograma	24

Anexo B: Presupuesto Estimado	25
Appendix C: Documentación del paquete de Python	26

RESUMEN

INTRODUCCIÓN Y OBJETIVOS

1.1 INTRODUCCIÓN

1.1.1 *Contextualización*

La Inteligencia Artificial (IA), impulsada por el avance en algoritmos de aprendizaje automático ha experimentado un rápido desarrollo debido a su reciente viabilidad de ejecución en hardware y la creciente disponibilidad de datos.

Sus fundamentos matemáticos, incluyendo el análisis, la probabilidad y el álgebra lineal que conforman la teoría del aprendizaje estadístico son esenciales para construir modelos que aprenden de los datos y realizan predicciones.

Como aplicación particular se encuentra la capacidad de estos algoritmos para identificar patrones en datos generados por las tecnologías de Internet Industrial de las Cosas (IoT), basadas en redes de sensores y dispositivos conectados a Internet que recopilan y transmiten datos en tiempo real.

1.1.2 *Descripción del problema*

El marco de este trabajo está relacionado con un proyecto de investigación de un sistema IoT implementado en la región de Poqueira, en la Alpujarra granadina [?]. El sistema IoT ha procesado la información recogida por los sensores visuales y la ha exportado a un conjunto de datos con múltiples características.

El entrenamiento de modelos de Aprendizaje Automático a partir de estos datos podría predecir variables que mejorasen la gestión de la ocupación de la zona, promoviendo así una mayor sostenibilidad en la comunidad local.

Sin embargo, la complejidad de la información obtenida, la presencia de ruido y la falta de datos etiquetados, dificultan la tarea. En este contexto, la generación de datos sintéticos se presenta como solución, pues permite añadir al conjunto de datos

original muestras generadas artificialmente que simulan las características de los datos reales. Esta técnica facilita el entrenamiento de modelos más robustos y precisos, abordando la escasez de etiquetas y mejorando la precisión de los modelos.

Aunque la literatura existente ha utilizado diversos Modelos de Generación Profunda (DGMs), incluyendo Redes Generativas Antagónicas (GANs) y Modelos de Autoencoders Variacionales (VAEs) [?], en este proyecto nos centraremos en los Modelos Transformers con Atención, pues su capacidad de modelar dependencias a largo plazo en los datos, los presenta como una alternativa prometedora.

1.1.3 Estructura del trabajo

Este trabajo se organiza en cinco partes, cada una de las cuales se divide en capítulos que abordan diferentes aspectos del problema y su solución. La estructura sigue las directrices establecidas para los trabajos de fin de grado.

- **Primera parte:** Presenta la introducción y los objetivos del proyecto.
- **Segunda parte:** Detalla los fundamentos matemáticos y teóricos del aprendizaje automático que sustentan el trabajo.
- **Tercera parte:** Se centra en el análisis de los modelos Transformer y perceptrones multicapa, explorando sus fundamentos matemáticos y arquitectura.
- **Cuarta parte:** Describe el desarrollo del proyecto y los experimentos realizados, incluyendo detalles sobre el problema, el software utilizado, el preprocesamiento de datos y las métricas de evaluación.
- **Quinta parte:** Presenta las conclusiones y reflexiones finales sobre el trabajo, así como recomendaciones para futuras investigaciones.

1.1.4 Herramientas Matemáticas e Informáticas

Para el desarrollo de este trabajo se utilizarán herramientas matemáticas e informáticas clave. Los fundamentos matemáticos incluyen análisis (diferenciabilidad), probabilidad y estadística para el manejo de la incertidumbre, y álgebra lineal, esenciales en el diseño y optimización de modelos de *machine learning*.

La implementación se llevará a cabo en Python, utilizando bibliotecas como *Numpy*, *Pandas*, y *Matplotlib* para el análisis de datos, y *Pypots* para el entrenamiento de los modelos *Transformers*.

1.1.5 *Bibliografía Fundamental*

Para la elaboración del Capítulo 3, Teoría del Aprendizaje Estadístico se ha destacamos el uso del libro Learning from Data [?].

1.2 OBJETIVOS

1.2.1 *Objetivo General*

El objetivo general de este trabajo es revisar los fundamentos matemáticos del aprendizaje automático y aplicar este conocimiento en la implementación de modelos generativos, específicamente utilizando arquitecturas Transformer.

1.2.2 *Parte de Matemáticas Específicos*

1. **Estudiar los fundamentos matemáticos del aprendizaje automático:** Revisar y comprender los conceptos matemáticos y teóricos que sustentan el aprendizaje automático, incluyendo teoría sobre cotas de generalización, optimización y regularización.

El objetivo 1 de matemáticas se abordará en el Capítulo 2.

1.2.3 *Parte de Informática*

1. **Definir la arquitectura Transformer. Modelo SAITS** Definir Encoder, Decoder, FFN, mecanismo de atención y como la arquitectura transformers los combina.
2. **Descripción de los conjuntos de datos. EDA** Llevar a cabo una descripción del conjunto de dato obtenido del sistema IoT de detección de vehículos. Realizar un análisis exploratorio de los datos.
3. **Entrenamiento mediante la biblioteca PyPOTS. Rendimiento** Evaluar el rendimiento y efectividad del modelo Transformer con atención a partir de métricas de evaluación, comparándolo con otros enfoques de modelado generativo.

El objetivo 1 de Informática se abordará en el Capítulo 3, el objetivo 2 en el Capítulo 4 y el objetivo 3 en el Capítulo 5.

FUNDAMENTOS MATEMÁTICOS DEL APRENDIZAJE AUTOMÁTICO

Esta sección establece las bases matemáticas necesarias para comprender los modelos computacionales utilizados, con especial énfasis en la manipulación de datos estructurados (matrices) y los principios de optimización que rigen su entrenamiento.

2.0.1 Álgebra Lineal: Matrices y Espacios Vectoriales

Las matrices son estructuras fundamentales para representar datos y parámetros en modelos computacionales, especialmente en redes neuronales y Transformers [?].

Definición 1 (Matriz). Una **matriz** $\mathbf{A}$ de tamaño $m \times n$, con entradas en un cuerpo $\mathbb{K}$ (típicamente $\mathbb{R}$ o $\mathbb{C}$), es una disposición rectangular de m filas y n columnas de elementos de $\mathbb{K}$. Se denota por $\mathbf{A} = (a_{ij})$ o $[\mathbf{A}]_{ij} = a_{ij}$, donde $a_{ij} \in \mathbb{K}$ es la entrada en la fila i y la columna j , para $1 \leq i \leq m$ y $1 \leq j \leq n$. El conjunto de todas estas matrices se denota $M_{m \times n}(\mathbb{K})$ o $\mathbb{K}^{m \times n}$.

Definición 2 (Igualdad de Matrices). Dos matrices $\mathbf{A} = (a_{ij})$ y $\mathbf{B} = (b_{ij})$ son **iguales** si pertenecen al mismo espacio $\mathbb{K}^{m \times n}$ (tienen el mismo tamaño) y $a_{ij} = b_{ij}$ para todos $1 \leq i \leq m, 1 \leq j \leq n$.

Definición 3 (Suma de Matrices). Dadas $\mathbf{A}, \mathbf{B} \in \mathbb{K}^{m \times n}$, su **suma** es la matriz $\mathbf{A} + \mathbf{B} \in \mathbb{K}^{m \times n}$ definida por $[\mathbf{A} + \mathbf{B}]_{ij} = a_{ij} + b_{ij}$.

Definición 4 (Producto por un Escalar). Dado $\mathbf{A} \in \mathbb{K}^{m \times n}$ y un escalar $c \in \mathbb{K}$, el **producto por un escalar** es la matriz $c\mathbf{A} \in \mathbb{K}^{m \times n}$ definida por $[c\mathbf{A}]_{ij} = c \cdot a_{ij}$.

Definición 5 (Espacio Vectorial de Matrices). El conjunto $\mathbb{K}^{m \times n}$, con las operaciones de suma (def:matrix_sum) y producto por un escalar (def: scalar_product), forma un **espacio vectorial** sobre $\mathbb{K}$ (con todas las entradas 0) actúa como vector nulo.

Proposición 1 (Axiomas del Espacio Vectorial $\mathbb{K}^{m \times n}$). Las operaciones definidas en $\mathbb{K}^{m \times n}$ satisfacen los axiomas de espacio vectorial para cualesquiera matrices $\mathbf{A}, \mathbf{B}, \mathbf{C} \in \mathbb{K}^{m \times n}$ y escalares $c, d \in \mathbb{K}$:

1. Asociatividad de la suma: $(\mathbf{A} + \mathbf{B}) + \mathbf{C} = \mathbf{A} + (\mathbf{B} + \mathbf{C})$
2. Conmutatividad de la suma: $\mathbf{A} + \mathbf{B} = \mathbf{B} + \mathbf{A}$
3. Existencia del elemento neutro (matriz cero $\mathbf{0}_{m \times n}$): $\mathbf{A} + \mathbf{0}_{m \times n} = \mathbf{A}$
4. Existencia del opuesto (matriz $-\mathbf{A} = (-a_{ij})$): $\mathbf{A} + (-\mathbf{A}) = \mathbf{0}_{m \times n}$
5. Asociatividad del producto por escalar: $(cd)\mathbf{A} = c(d\mathbf{A})$
6. Distributividad respecto a la suma de escalares: $(c + d)\mathbf{A} = c\mathbf{A} + d\mathbf{A}$
7. Distributividad respecto a la suma de matrices: $c(\mathbf{A} + \mathbf{B}) = c\mathbf{A} + c\mathbf{B}$
8. Existencia del elemento neutro para el producto por escalar (el escalar 1): $1 \cdot \mathbf{A} = \mathbf{A}$

Demostración. Cada axioma se verifica fácilmente a partir de la definición de las operaciones y las propiedades correspondientes en el cuerpo $\mathbb{K}$. Por ejemplo, para la conmutatividad de la suma (Axioma 2): Sean $\mathbf{A} = (a_{ij})$ y $\mathbf{B} = (b_{ij})$. $[\mathbf{A} + \mathbf{B}]_{ij} = a_{ij} + b_{ij}$ (por def:matrix_sum). Como la suma en $\mathbb{K}$ es conmutativa, $a_{ij} + b_{ij} = b_{ij} + a_{ij}$. Entonces, $[\mathbf{B} + \mathbf{A}]_{ij} = b_{ij} + a_{ij}$ (también por def:matrix_sum). Por lo tanto, se tiene que $[\mathbf{A} + \mathbf{B}]_{ij} = [\mathbf{B} + \mathbf{A}]_{ij}$ para todo i, j , lo que implica $\mathbf{A} + \mathbf{B} = \mathbf{B} + \mathbf{A}$ (por def:matrix_equality). Las demostraciones para los demás

Definición 6 (Producto Matricial). El **producto matricial** de $\mathbf{A} \in \mathbb{K}^{m \times p}$ y $\mathbf{B} \in \mathbb{K}^{p \times n}$ es la matriz $\mathbf{C} = \mathbf{AB} \in \mathbb{K}^{m \times n}$, definida por:

$$[\mathbf{AB}]_{ij} = c_{ij} = \sum_{k=1}^p a_{ik}b_{kj}$$

para $1 \leq i \leq m$ y $1 \leq j \leq n$. *Nota:* El producto $\mathbf{AB}$ está definido solo si el número de columnas de $\mathbf{A}$ es igual al número de filas de $\mathbf{B}$.

Proposición 2 (Propiedades del Producto Matricial). Sean $\mathbf{A}, \mathbf{B}, \mathbf{C}$ matrices de tamaños compatibles y c un escalar:

1. Asociatividad: $(\mathbf{AB})\mathbf{C} = \mathbf{A}(\mathbf{BC})$
2. Distributividad: $\mathbf{A}(\mathbf{B} + \mathbf{C}) = \mathbf{AB} + \mathbf{AC}$ y $(\mathbf{A} + \mathbf{B})\mathbf{C} = \mathbf{AC} + \mathbf{BC}$
3. Compatibilidad escalar: $c(\mathbf{AB}) = (c\mathbf{A})\mathbf{B} = \mathbf{A}(c\mathbf{B})$
4. No Conmutatividad general: $\mathbf{AB} \neq \mathbf{BA}$ en general.

Demostración de la Asociatividad (Prop. ??1). Sean $\mathbf{A} \in \mathbb{K}^{m \times p}$, $\mathbf{B} \in \mathbb{K}^{p \times q}$ y $\mathbf{C} \in \mathbb{K}^{q \times n}$. El producto $\mathbf{AB}$ es una matriz de $m \times q$ con entradas $[\mathbf{AB}]_{ik} = \sum_{l=1}^p a_{il}b_{lk}$. El producto $(\mathbf{AB})\mathbf{C}$ es una matriz de $m \times n$ con entradas:

$$[(\mathbf{AB})\mathbf{C}]_{ij} = \sum_{k=1}^q [\mathbf{AB}]_{ik}c_{kj} = \sum_{k=1}^q \left(\sum_{l=1}^p a_{il}b_{lk} \right) c_{kj} = \sum_{k=1}^q \sum_{l=1}^p a_{il}b_{lk}c_{kj}$$

El producto $\mathbf{BC}$ es una matriz de $p \times n$ con entradas $[\mathbf{BC}]_{lj} = \sum_{k=1}^q b_{lk}c_{kj}$. El producto $\mathbf{A}(\mathbf{BC})$ es una matriz de $m \times n$ con entradas:

$$[\mathbf{A}(\mathbf{BC})]_{ij} = \sum_{l=1}^p a_{il}[\mathbf{BC}]_{lj} = \sum_{l=1}^p a_{il} \left(\sum_{k=1}^q b_{lk}c_{kj} \right) = \sum_{l=1}^p \sum_{k=1}^q a_{il}b_{lk}c_{kj}$$

Dado que la suma finita es conmutativa y asociativa en $\mathbb{K}$, podemos intercambiar el orden de las sumas: $\sum_{k=1}^q \sum_{l=1}^p a_{il}b_{lk}c_{kj} = \sum_{l=1}^p \sum_{k=1}^q a_{il}b_{lk}c_{kj}$. Por lo tanto, $[(\mathbf{AB})\mathbf{C}]_{ij} = [\mathbf{A}(\mathbf{BC})]_{ij}$ para todo i, j , lo que implica $(\mathbf{AB})\mathbf{C} = \mathbf{A}(\mathbf{BC})$. $\square$

Definición 7 (Producto Elemento a Elemento - Hadamard). Dadas $\mathbf{A}, \mathbf{B} \in \mathbb{K}^{m \times n}$ (mis-mo tamaño), su **producto elemento a elemento** o **producto de Hadamard** es la matriz $\mathbf{A} \odot \mathbf{B} \in \mathbb{K}^{m \times n}$ definida por:

$$[\mathbf{A} \odot \mathbf{B}]_{ij} = a_{ij} \cdot b_{ij}$$

Definición 8 (Matriz Traspuesta). Dada $\mathbf{A} \in \mathbb{K}^{m \times n}$, su **matriz traspuesta**, $\mathbf{A}^T \in \mathbb{K}^{n \times m}$, se define por:

$$[\mathbf{A}^T]_{ij} = a_{ji}$$

Proposición 3 (Propiedades de la Traspuesta). Sean $\mathbf{A}, \mathbf{B}$ matrices de tamaños compatibles y c un escalar:

1. $(\mathbf{A}^T)^T = \mathbf{A}$
2. $(\mathbf{A} + \mathbf{B})^T = \mathbf{A}^T + \mathbf{B}^T$
3. $(c\mathbf{A})^T = c\mathbf{A}^T$
4. $(\mathbf{AB})^T = \mathbf{B}^T\mathbf{A}^T$

Demostración de la Traspuesta de un Producto (Prop. ??4). Sean $\mathbf{A} \in \mathbb{K}^{m \times p}$ y $\mathbf{B} \in \mathbb{K}^{p \times n}$. El producto $\mathbf{AB}$ es de $m \times n$. Su traspuesta $(\mathbf{AB})^T$ es de $n \times m$. La entrada (i, j) de $(\mathbf{AB})^T$ es la entrada (j, i) de $\mathbf{AB}$:

$$[(\mathbf{AB})^T]_{ij} = [\mathbf{AB}]_{ji} = \sum_{k=1}^p a_{jk}b_{ki}$$

Ahora consideremos $\mathbf{B}^T \in \mathbb{K}^{n \times p}$ y $\mathbf{A}^T \in \mathbb{K}^{p \times m}$. El producto $\mathbf{B}^T\mathbf{A}^T$ es de $n \times m$. La entrada (i, j) de $\mathbf{B}^T\mathbf{A}^T$ es:

$$[\mathbf{B}^T\mathbf{A}^T]_{ij} = \sum_{k=1}^p [\mathbf{B}^T]_{ik}[\mathbf{A}^T]_{kj}$$

Por la def:transpose, $[\mathbf{B}^T]_{ik} = b_{ki}$ y $[\mathbf{A}^T]_{kj} = a_{jk}$. Entonces, $[\mathbf{B}^T\mathbf{A}^T]_{ij} = \sum_{k=1}^p b_{ki}a_{jk}$. Como la multiplicación y suma en $\mathbb{K}$ permiten reordenar los términos del producto y la suma: $\sum_{k=1}^p b_{ki}a_{jk} = \sum_{k=1}^p a_{jk}b_{ki}$. Por lo tanto, $[(\mathbf{AB})^T]_{ij} = [\mathbf{B}^T\mathbf{A}^T]_{ij}$ para todo i, j , lo que implica $(\mathbf{AB})^T = \mathbf{B}^T\mathbf{A}^T$. $\square$

2.0.2 Estadística Descriptiva: Medidas de Centralización

Definición 9 (Media Aritmética). Dada una colección $\{x_1, \dots, x_N\} \subset \mathbb{R}$, la **media aritmética** es $\bar{x} = \frac{1}{N} \sum_{i=1}^N x_i$.

Definición 10 (Mediana). Dada una colección $\{x_1, \dots, x_N\} \subset \mathbb{R}$, y su versión ordenada $x_{(1)} \leq \dots \leq x_{(N)}$, la **mediana** Me es $x_{((N+1)/2)}$ si N es impar, o $\frac{1}{2}(x_{(N/2)} + x_{(N/2+1)})$ si N es par.

2.0.3 Interpolación Lineal

Definición 11 (Interpolación Lineal). Dados (x_0, y_0) y (x_1, y_1) con $x_0 \neq x_1$, la función de **interpolación lineal** para $x \in [x_0, x_1]$ es:

$$f(x) = y_0 + \frac{x - x_0}{x_1 - x_0}(y_1 - y_0) = (1 - \alpha)y_0 + \alpha y_1$$

donde $\alpha = (x - x_0)/(x_1 - x_0)$.

2.0.4 Cálculo Multivariable y Optimización

El entrenamiento de modelos complejos se basa en minimizar una función objetivo utilizando métodos de optimización basados en el gradiente [?].

Definición 12 (Derivada Parcial). Sea $f : U \subseteq \mathbb{R}^n \rightarrow \mathbb{R}$ definida en un abierto U . La **derivada parcial** de f respecto a x_i en $\mathbf{a} \in U$ es:

$$\frac{\partial f}{\partial x_i}(\mathbf{a}) = \lim_{h \rightarrow 0} \frac{f(\mathbf{a} + h\mathbf{e}_i) - f(\mathbf{a})}{h}$$

donde $\mathbf{e}_i$ es el i -ésimo vector canónico.

Definición 13 (Gradiente). Sea $f : U \subseteq \mathbb{R}^n \rightarrow \mathbb{R}$ diferenciable en $\mathbf{a} \in U$. El **vector gradiente** $\nabla f(\mathbf{a}) \in \mathbb{R}^n$ es:

$$\nabla f(\mathbf{a}) = \left(\frac{\partial f}{\partial x_1}(\mathbf{a}), \dots, \frac{\partial f}{\partial x_n}(\mathbf{a}) \right)^T$$

El gradiente $\nabla f(\mathbf{a})$ apunta en la dirección de máximo aumento local de f .

Definición 14 (Matriz Jacobiana). Sea $\mathbf{f} : U \subseteq \mathbb{R}^n \rightarrow \mathbb{R}^m$, $\mathbf{f}(\mathbf{x}) = (f_1(\mathbf{x}), \dots, f_m(\mathbf{x}))$. Si las derivadas parciales existen en $\mathbf{x} \in U$, la **matriz Jacobiana** $\mathbf{J}_{\mathbf{f}}(\mathbf{x}) \in \mathbb{R}^{m \times n}$ es:

$$[\mathbf{J}_{\mathbf{f}}(\mathbf{x})]_{ij} = \frac{\partial f_i}{\partial x_j}(\mathbf{x})$$

Notar que si $m = 1$, entonces $\mathbf{J}_f(\mathbf{x}) = (\nabla f(\mathbf{x}))^T$.

Teorema 1 (Regla de la Cadena Multivariable). Sean $\mathbf{g} : U \subseteq \mathbb{R}^n \rightarrow V \subseteq \mathbb{R}^m$ diferenciable en $\mathbf{x}_0$, y $\mathbf{f} : V \rightarrow \mathbb{R}^p$ diferenciable en $\mathbf{y}_0 = \mathbf{g}(\mathbf{x}_0)$. La composición $\mathbf{h} = \mathbf{f} \circ \mathbf{g} : U \rightarrow \mathbb{R}^p$ es diferenciable en $\mathbf{x}_0$, y su Jacobiana es:

$$\mathbf{J}_h(\mathbf{x}_0) = \mathbf{J}_f(\mathbf{y}_0)\mathbf{J}_g(\mathbf{x}_0)$$

Esta regla es fundamental para el algoritmo de retropropagación (backpropagation) [?].

2.0.5 Funciones de Activación: Softmax

La función Softmax es crucial en modelos de clasificación multiclase y en mecanismos de atención [?].

Definición 15 (Función Softmax). La función $\sigma : \mathbb{R}^K \rightarrow \mathbb{R}^K$ toma $\mathbf{z} \in \mathbb{R}^K$ y devuelve $\sigma(\mathbf{z})$ con componentes:

$$[\sigma(\mathbf{z})]_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

Proposición 4 (Propiedades de Softmax). Para $\mathbf{p} = \sigma(\mathbf{z})$:

1. Positividad: $p_i > 0$ para todo $i = 1, \dots, K$.
2. Normalización: $\sum_{i=1}^K p_i = 1$.

(Las salidas forman una distribución de probabilidad discreta).

Demostración de la Propiedad ??.

$$\sum_{i=1}^K [\sigma(\mathbf{z})]_i = \sum_{i=1}^K \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}} = \frac{1}{\sum_{j=1}^K e^{z_j}} \sum_{i=1}^K e^{z_i} = \frac{\sum_{i=1}^K e^{z_i}}{\sum_{j=1}^K e^{z_j}} = 1$$

La propiedad 1 es evidente ya que la función exponencial e^x es siempre positiva. $\square$

Proposición 5 (Jacobiana de Softmax). La Jacobiana de σ en $\mathbf{z}$ es $\mathbf{J}_\sigma(\mathbf{z}) \in \mathbb{R}^{K \times K}$ con entradas:

$$[\mathbf{J}_\sigma(\mathbf{z})]_{ij} = \frac{\partial \sigma_i}{\partial z_j} = \sigma_i(\mathbf{z})(\delta_{ij} - \sigma_j(\mathbf{z}))$$

donde δ_{ij} es la delta de Kronecker. Matricialmente: $\mathbf{J}_\sigma(\mathbf{z}) = \text{diag}(\sigma(\mathbf{z})) - \sigma(\mathbf{z})\sigma(\mathbf{z})^T$.

Demostración. Consideremos dos casos: $i = j$ y $i \neq j$. Sea $S = \sum_{k=1}^K e^{z_k}$. Entonces $\sigma_i(\mathbf{z}) = e^{z_i} / S$.

Caso 1: $i = j$. Aplicando la regla del cociente para derivadas parciales:

$$\begin{aligned}\frac{\partial \sigma_i(\mathbf{z})}{\partial z_i} &= \frac{\partial}{\partial z_i} \left(\frac{e^{z_i}}{S} \right) = \frac{(\frac{\partial}{\partial z_i} e^{z_i})S - e^{z_i}(\frac{\partial S}{\partial z_i})}{S^2} \\ &= \frac{e^{z_i}S - e^{z_i}e^{z_i}}{S^2} \quad (\text{ya que } \frac{\partial S}{\partial z_i} = \frac{\partial}{\partial z_i} \sum_k e^{z_k} = e^{z_i}) \\ &= \frac{e^{z_i}}{S} - \left(\frac{e^{z_i}}{S} \right)^2 = \sigma_i(\mathbf{z}) - (\sigma_i(\mathbf{z}))^2 \\ &= \sigma_i(\mathbf{z})(1 - \sigma_i(\mathbf{z})) = \sigma_i(\mathbf{z})(\delta_{ii} - \sigma_i(\mathbf{z}))\end{aligned}$$

Caso 2: $i \neq j$. Aplicando la regla del cociente:

$$\begin{aligned}\frac{\partial \sigma_i(\mathbf{z})}{\partial z_j} &= \frac{\partial}{\partial z_j} \left(\frac{e^{z_i}}{S} \right) = \frac{(\frac{\partial}{\partial z_j} e^{z_i})S - e^{z_i}(\frac{\partial S}{\partial z_j})}{S^2} \\ &= \frac{0 \cdot S - e^{z_i}e^{z_j}}{S^2} \quad (\text{ya que } \frac{\partial S}{\partial z_j} = \frac{\partial}{\partial z_j} \sum_k e^{z_k} = e^{z_j}) \\ &= -\frac{e^{z_i}e^{z_j}}{S^2} = -\frac{e^{z_i}}{S} \frac{e^{z_j}}{S} = -\sigma_i(\mathbf{z})\sigma_j(\mathbf{z}) \\ &= \sigma_i(\mathbf{z})(0 - \sigma_j(\mathbf{z})) = \sigma_i(\mathbf{z})(\delta_{ij} - \sigma_j(\mathbf{z}))\end{aligned}$$

Ambos casos verifican la fórmula dada. □

2.1 SERIES TEMPORALES Y DATOS FALTANTES: FORMALIZACIÓN MATRICIAL

Las series temporales, secuencias de observaciones ordenadas cronológicamente, a menudo presentan valores faltantes. Utilizamos las definiciones de álgebra lineal (subsec:linear_algebra) para formalizar estos conceptos.

Definición 16 (Serie Temporal Multivariante Matricial). Una serie temporal multivariante se representa como una matriz $\mathbf{X} \in \mathbb{R}^{T \times D}$ (def:matrix), donde T es el número de pasos de tiempo (filas) y D es el número de variables (columnas). La fila i de $\mathbf{X}$, denotada $\mathbf{x}(t_i)^T$ o $\mathbf{X}_{i,:}$, representa el vector de observaciones en el tiempo $t_i \in \mathcal{T} = \{t_1, \dots, t_T\}$. La columna j , denotada $\mathbf{X}_{:,j}$, representa la serie temporal univariante para la variable j . Desde una perspectiva probabilística, $\mathbf{X}$ es una realización de un proceso estocástico $\{\mathbf{X}(t)\}_{t \in \mathcal{T}}$.

Definición 17 (Matriz de Máscara de Observación). La disponibilidad de los datos en $\mathbf{X}$ se representa mediante una **matriz de máscara** $\mathbf{M} \in \{0, 1\}^{T \times D}$ (def:matrix), con las mismas dimensiones que $\mathbf{X}$, tal que:

$$[\mathbf{M}]_{ij} = M_{ij} = \begin{cases} 1, & \text{si } [\mathbf{X}]_{ij} = x_j(t_i) \text{ es observado} \\ 0, & \text{si } [\mathbf{X}]_{ij} = x_j(t_i) \text{ está faltante} \end{cases}$$

Esta convención es común en la literatura reciente [? ?].

Definición 18 (Serie Temporal Parcialmente Observada (Tupla)). Una serie temporal parcialmente observada se representa formalmente mediante la tupla $(\mathbf{X}, \mathbf{M})$, donde $\mathbf{X}$ es la matriz de datos y $\mathbf{M}$ es la máscara (def:matrix_{mask}) que indica explícitamente la ubicación de las observaciones.

Comentario 1. Una forma útil de manipular y aislar los valores observados es mediante el producto de Hadamard (def:hadamard_{product}) con la matriz de máscara $\mathbf{M}$: $\mathbf{X}_{\text{obs_masked}} = \mathbf{X} \odot \mathbf{M}$. La matriz resultante, $\mathbf{X}_{\text{obs_masked}}$, contiene los valores observados $[\mathbf{X}]_{ij}$ en las posiciones donde $[\mathbf{M}]_{ij} = 1$, y ceros en las posiciones donde $[\mathbf{M}]_{ij} = 0$. Esta operación facilita la separación conceptual y a veces computacional de los datos. Recordemos que los conjuntos de índices observados y faltantes son $\mathcal{O} = \{(i, j) | M_{ij} = 1\}$ y $\mathcal{M} = \{(i, j) | M_{ij} = 0\}$, y los conjuntos de valores correspondientes son $\mathbf{X}_{\text{obs}} = \{x_j(t_i) | (i, j) \in \mathcal{O}\}$ y $\mathbf{X}_{\text{mis}} = \{x_j(t_i) | (i, j) \in \mathcal{M}\}$.

2.1.1 Fundamentos de Probabilidad para Mecanismos de Datos Faltantes

Para comprender formalmente los mecanismos que generan datos faltantes, es necesario revisar algunos conceptos básicos de la teoría de la probabilidad, particularmente la probabilidad condicionada y la independencia estadística. En este contexto, trataremos la matriz de datos completa (hipotética) $\mathbf{X}$ y la matriz de máscara $\mathbf{M}$ como realizaciones de procesos aleatorios o, más simplemente, como matrices aleatorias.

Definición 19 (Espacio de Probabilidad y Eventos). Un **espacio de probabilidad** es una terna $(\Omega, \mathcal{F}, P)$, donde Ω es el **espacio muestral** (el conjunto de todos los posibles resultados), $\mathcal{A}$ es una σ -álgebra de **eventos** (subconjuntos de Ω cuya probabilidad podemos medir), y $P : \mathcal{A} \rightarrow [0, 1]$ es una **medida de probabilidad** que satisface los axiomas de Kolmogorov (no negatividad, normalización $P(\Omega) = 1$, y σ -aditividad). Un evento $A \in \mathcal{A}$ es un conjunto de resultados de interés.

Definición 20 (Variables Aleatorias). Una **variable aleatoria** (v.a.) es una función medible $X : \Omega \rightarrow \mathbb{R}$ (o a $\mathbb{R}^n$ para un vector aleatorio, o a $\mathbb{K}^{m \times n}$ para una matriz aleatoria). Asocia un valor numérico (o vectorial/matricial) a cada resultado del espacio muestral. En nuestro caso, podemos considerar cada entrada X_{ij} de la matriz de datos $\mathbf{X}$ y cada entrada M_{ij} de la matriz de máscara $\mathbf{M}$ como variables aleatorias.

Definición 21 (Probabilidad de un Evento). La **probabilidad** de un evento A , denotada $P(A)$, cuantifica la verosimilitud de que el resultado del experimento aleatorio pertenezca al conjunto A . Por ejemplo, $P(M_{ij} = 0)$ representa la probabilidad de que la entrada (i, j) de la matriz de datos esté faltante.

Definición 22 (Probabilidad Condicionada). Dados dos eventos A y B con $P(B) > 0$, la **probabilidad condicionada** de A dado B , denotada $P(A|B)$, se define como:

$$P(A|B) = \frac{P(A \cap B)}{P(B)}$$

Representa la probabilidad de que ocurra el evento A *sabiendo* que el evento B ya ha ocurrido. En el contexto de datos faltantes, $P(M_{ij} = 0|\mathbf{X})$ representa la probabilidad de que la entrada (i, j) falte, condicionada al conocimiento de *todos* los valores (observados y faltantes) de la matriz de datos $\mathbf{X}$. Similarmente, $P(M_{ij} = 0|\mathbf{X}_{\text{obs}})$ condiciona únicamente sobre el subconjunto de valores observados $\mathbf{X}_{\text{obs}} = \{x_k(t_l) | M_{lk} = 1\}$.

Definición 23 (Independencia Estadística). Dos eventos A y B son **estadísticamente independientes** si la ocurrencia de uno no afecta la probabilidad del otro. Formalmente, esto ocurre si $P(A \cap B) = P(A)P(B)$. Si $P(B) > 0$, una definición equivalente es:

$$P(A|B) = P(A)$$

De manera análoga, dos variables (o conjuntos de variables) aleatorias X e Y son independientes si el conocimiento de Y no altera la distribución de probabilidad de X .

Estos conceptos son la base para clasificar los mecanismos de datos faltantes. La diferencia entre MCAR, MAR y MNAR radica precisamente en la dependencia (o independencia) probabilística entre el evento de que un dato falte ($M_{ij} = 0$) y los valores de la propia serie temporal ($\mathbf{X}$, $\mathbf{X}_{\text{obs}}$, o $\mathbf{X}_{\text{mis}}$), como se detalla a continuación.

MECANISMOS DE DATOS FALTANTES. La comprensión del mecanismo que genera los datos faltantes es un paso crucial antes de seleccionar una estrategia de imputación, ya que diferentes mecanismos tienen distintas implicaciones para la validez de los análisis y la idoneidad de los métodos [?]. La clasificación fundamental, propuesta originalmente por Rubin [?], se basa en la relación de dependencia probabilística entre la ausencia de un dato (el evento $M_{ij} = 0$) y los valores de la propia serie temporal ($\mathbf{X}$), utilizando el concepto de probabilidad condicionada (def:conditional_{prob})[?]. *Se distinguen tres mecanismos principales :*

- **MCAR (Missing Completely At Random - Faltantes Completamente al Azar):**
 - *Definición Formal:* La probabilidad de que un dato falte es estadísticamente independiente (def:independence) del valor de *cualquier* variable en la serie temporal, ya sea observada o faltante.

$$P(M_{ij} = 0|\mathbf{X}) = P(M_{ij} = 0)$$

- *Intuición:* La ausencia es un suceso puramente aleatorio, como si se lanzara una moneda para decidir si borrar o no cada entrada, o si fallos técnicos completamente aleatorios (no relacionados con las mediciones) causaran pérdidas de datos.

- *Implicaciones:* Es el supuesto más fuerte y, a menudo, poco realista. Sin embargo, si se cumple, los datos observados $\mathbf{X}_{\text{obs}}$ son una submuestra completamente representativa de la serie completa hipotética. Los análisis realizados únicamente sobre los casos completos (listwise deletion) producirán estimaciones insesgadas de los parámetros (aunque pueden ser ineficientes por la pérdida de datos). Muchos métodos de imputación funcionan bien bajo MCAR.

■ **MAR (Missing At Random - Faltantes al Azar):**

- *Definición Formal:* La probabilidad de que un dato falte puede depender de los valores de las variables *observadas*, pero, una vez que se tienen en cuenta estas variables observadas, ya no depende de los valores de las variables *faltantes*.

$$P(M_{ij} = 0|\mathbf{X}) = P(M_{ij} = 0|\mathbf{X}_{\text{obs}})$$

Es decir, la ausencia $M_{ij} = 0$ es condicionalmente independiente de $\mathbf{X}_{\text{mis}}$, dado $\mathbf{X}_{\text{obs}}$.

- *Intuición:* La ausencia no es completamente aleatoria, sino que está relacionada con otras características que sí conocemos. Si conocemos esas características, la razón por la que falta el dato específico ya está explicada y no depende del valor que no vemos.
- *Implicaciones:* Es un supuesto más débil y plausible que MCAR en muchas situaciones reales. Los análisis sobre casos completos pueden producir estimaciones sesgadas. Sin embargo, los métodos de imputación basados en modelos (como la imputación múltiple o métodos basados en máxima verosimilitud) que utilicen correctamente la información de $\mathbf{X}_{\text{obs}}$ (incluyendo las variables que predicen la ausencia) pueden, en teoría, producir resultados insesgados. La clave es que la información sobre la ausencia está contenida en los datos observados.

■ **MNAR (Missing Not At Random - Faltantes No al Azar):**

- *Definición Formal:* La probabilidad de que un dato falte depende del valor de la propia variable faltante, incluso después de tener en cuenta todas las variables observadas.

$$P(M_{ij} = 0|\mathbf{X}) \text{ depende de } \mathbf{X}_{\text{mis}}$$

(La condición $P(M_{ij} = 0|\mathbf{X}) = P(M_{ij} = 0|\mathbf{X}_{\text{obs}})$ no se cumple).

- *Intuición:* La razón por la que falta el dato está directamente relacionada con el valor que no estamos viendo. El propio valor faltante es informativo sobre por qué falta.
- *Implicaciones:* Es el escenario más complejo y problemático. Casi todos los métodos estándar de manejo de datos faltantes (listwise deletion, imputación simple, imputación múltiple estándar, modelos ML que ignoran el mecanismo) producen estimaciones sesgadas. Abordar datos MNAR requiere técnicas más avanzadas que modelen explícitamente el mecanismo de ausencia (modelos de selección, modelos de mezcla de patrones) o rea-

lizar análisis de sensibilidad para evaluar el impacto potencial del mecanismo MNAR. La información sobre la ausencia no está completamente contenida en los datos observados.

Es importante destacar que la distinción entre MAR y MNAR no siempre puede verificarse empíricamente solo con los datos observados; a menudo requiere supuestos basados en el conocimiento del dominio o el diseño del estudio. Sin embargo, comprender estas categorías es fundamental para evaluar críticamente los resultados de cualquier análisis realizado en presencia de datos faltantes y para justificar la elección de una técnica de imputación particular. Los métodos de imputación más sofisticados, como los basados en aprendizaje profundo (ej. SAITS), intentan capturar las complejas dependencias en los datos que podrían estar relacionadas con los mecanismos MAR o incluso, implícitamente, con algunos patrones MNAR, aunque la garantía teórica bajo MNAR es más difícil de establecer [? ?].

2.2 FUNDAMENTOS MATEMÁTICOS DEL ENTRENAMIENTO DEL TRANSFORMER

El entrenamiento de un Transformer [?], como modelo predictivo f_θ (eq:matrix_predictive;imputation) utilizando descenso de gradiente.

- **Parámetros θ :** Incluyen matrices de pesos $\mathbf{W}$ y vectores de sesgo $\mathbf{b}$ (subsec:linear_algebra). **Función** $= f_\theta(\mathbf{X})$ y la referencia $\mathbf{Y}$ (ej. MSE usando normas matriciales).
- **Descenso de Gradiente:** Actualiza $\theta^{(t+1)} = \theta^{(t)} - \eta \nabla L(\theta^{(t)})$.
- **Gradiente $\nabla L(\theta)$:** Es la colección de derivadas parciales $\frac{\partial L}{\partial w_{ij}}, \frac{\partial L}{\partial b_i}$ para todos los parámetros. Se calcula mediante **Backpropagation**, que aplica eficientemente la Regla de la Cadena (thm:chain_rule) a través de las operaciones del modelo (producto matricial, suma, Softmax, etc.). El proceso requiere herramientas del cálculo diferencial multivariable (subsec:multivariable_calculus).

2.3 MÉTRICAS DE EVALUACIÓN PARA IMPUTACIÓN

Para cuantificar el rendimiento de los diferentes métodos de imputación en el contexto experimental descrito (def:exp_imputation_problem_revised), se utilizan métricas de error estándar. Estas métricas se calculan con los valores verdaderos originales $\mathcal{X}_{\text{orig}}^{\text{test}}$ únicamente en las posiciones donde los datos fueron artificialmente eliminados, según lo indicado por la máscara $\mathbf{M}_{\text{indicating}}^{\text{test}}$ (o equivalentemente $\mathbf{M}_{\text{artificial}}^{\text{test}}$).

Sea $\hat{\mathbf{X}} = \hat{\mathbf{X}}^{\text{test}} \in \mathbb{R}^{N \times T \times D}$ el tensor de valores imputados para el conjunto de test. Sea $\mathbf{X}_{\text{orig}} = \mathcal{X}_{\text{orig}}^{\text{test}} \in \mathbb{R}^{N \times T \times D}$ el tensor de valores verdaderos originales (ground truth) para el conjunto de test. Sea $\mathbf{M}_{\text{ind}} = \mathbf{M}_{\text{artificial}}^{\text{test}} \in \{0, 1\}^{N \times T \times D}$ la máscara que indica las posiciones artificialmente eliminadas (1 si fue eliminada, 0 si no).

Denotamos $[\hat{\mathcal{X}}]_{ktd}$ y $[\mathbf{X}_{\text{orig}}]_{ktd}$ a los valores en la muestra k , tiempo t ($1 \leq t \leq T$), y característica d ($1 \leq d \leq D$). $[\mathbf{M}_{\text{ind}}]_{ktd}$ es el valor correspondiente de la máscara.

Las métricas se calculan promediando sobre todas las muestras N y todas las posiciones indicadas por la máscara $\mathbf{M}_{\text{ind}}$.

Definición 24 (Error Absoluto Medio (MAE)). El MAE mide el promedio de los errores absolutos entre los valores imputados y los verdaderos en las posiciones indicadas:

$$\text{MAE}(\hat{\mathcal{X}}, \mathbf{X}_{\text{orig}}, \mathbf{M}_{\text{ind}}) = \frac{\sum_{k=1}^N \sum_{t=1}^T \sum_{d=1}^D |[\hat{\mathcal{X}}]_{ktd} - [\mathbf{X}_{\text{orig}}]_{ktd}| \cdot [\mathbf{M}_{\text{ind}}]_{ktd}}{\sum_{k=1}^N \sum_{t=1}^T \sum_{d=1}^D [\mathbf{M}_{\text{ind}}]_{ktd}} \quad (1)$$

donde el denominador representa el número total de puntos artificialmente eliminados.

Definición 25 (Raíz del Error Cuadrático Medio (RMSE)). El RMSE es la raíz cuadrada del promedio de los errores cuadráticos, penalizando más los errores grandes:

$$\text{RMSE}(\hat{\mathcal{X}}, \mathbf{X}_{\text{orig}}, \mathbf{M}_{\text{ind}}) = \sqrt{\frac{\sum_{k=1}^N \sum_{t=1}^T \sum_{d=1}^D ([\hat{\mathcal{X}}]_{ktd} - [\mathbf{X}_{\text{orig}}]_{ktd})^2 \cdot [\mathbf{M}_{\text{ind}}]_{ktd}}{\sum_{k=1}^N \sum_{t=1}^T \sum_{d=1}^D [\mathbf{M}_{\text{ind}}]_{ktd}}} \quad (2)$$

El término dentro de la raíz cuadrada es el Error Cuadrático Medio (MSE).

Definición 26 (Error Relativo Medio (MRE)). El MRE mide el error absoluto promedio relativo al valor verdadero original:

$$\text{MRE}(\hat{\mathcal{X}}, \mathbf{X}_{\text{orig}}, \mathbf{M}_{\text{ind}}) = \frac{\sum_{k=1}^N \sum_{t=1}^T \sum_{d=1}^D |[\hat{\mathcal{X}}]_{ktd} - [\mathbf{X}_{\text{orig}}]_{ktd}| \cdot [\mathbf{M}_{\text{ind}}]_{ktd}}{\sum_{k=1}^N \sum_{t=1}^T \sum_{d=1}^D |[\mathbf{X}_{\text{orig}}]_{ktd}| \cdot [\mathbf{M}_{\text{ind}}]_{ktd} + \epsilon} \quad (3)$$

Nota: Se añade un pequeño valor $\epsilon > 0$ al denominador para evitar la división por cero en caso de que algún valor verdadero original $[\mathbf{X}_{\text{orig}}]_{ktd}$ sea cero en una posición indicada por la máscara. La fórmula original de la imagen parece usar el producto Hadamard de forma ligeramente distinta, pero esta formulación (dividir la suma de errores absolutos por la suma de valores absolutos verdaderos, ambos enmascarados) es una interpretación común y robusta del MRE en este contexto.

DEL TRANSFORMER A SAITS: ARQUITECTURA Y APLICACIÓN

3.0.1 *Introducción a los Modelos Transformer*

Los modelos Transformer irrumpieron en el panorama del aprendizaje automático como una auténtica disrupción, marcando un antes y un después en el procesamiento de secuencias, especialmente en el ámbito del lenguaje natural (NLP). Su concepción representó un avance paradigmático, superando las limitaciones inherentes a las arquitecturas recurrentes (RNNs) y convolucionales (CNNs) que hasta entonces dominaban el campo. La esencia de su revolución reside en dos pilares fundamentales: su capacidad intrínseca para el **procesamiento en paralelo** y la introducción de un mecanismo de atención sumamente potente y flexible. Mientras que las RNNs procesaban la información de manera secuencial, limitando la eficiencia y la capacidad de capturar dependencias a largo alcance, los Transformers abordan la secuencia de entrada como un todo, operando simultáneamente sobre todos sus elementos. Este enfoque, orquestado por el mecanismo de atención, dota al modelo de la habilidad de discernir y ponderar la importancia relativa de cada parte de la entrada con respecto a las demás. En lugar de procesar la información paso a paso, el Transformer evalúa la relevancia contextual de cada elemento, optimizando el flujo de información y, en última instancia, potenciando significativamente su capacidad de aprendizaje y comprensión. Este capítulo se adentra en el corazón de la arquitectura Transformer, desglosando sus componentes esenciales y trazando un camino que culmina en SAITS (Self-Attention-based Imputation for Time Series), un modelo especializado que adapta la potencia del Transformer a la imputación de datos faltantes en series temporales. A través de este recorrido, demostraremos la versatilidad y el alcance de esta arquitectura, extendiéndose mucho más allá de sus orígenes en el procesamiento del lenguaje natural.

3.0.2 *Mecanismo de Atención Escalonada (Scaled Dot-Product Attention)*

El mecanismo de atención constituye el núcleo operativo y la innovación distintiva de los Transformers, siendo el principal artífice de su excepcional rendimiento. Den-

tro del espectro de mecanismos de atención existentes, la **atención escalonada por producto punto** (Scaled Dot-Product Attention) emerge como la piedra angular, el bloque constructivo esencial. Este ingenioso mecanismo habilita al modelo para discernir y cuantificar la similitud o relevancia entre diferentes posiciones dentro de la secuencia de entrada. De esta forma, determina con precisión qué fragmentos de la información entrante merecen una mayor ponderación al procesar cada elemento en particular. En términos prácticos, la atención confiere al modelo la capacidad de enfocar su "mirada" selectivamente sobre la información más crucial y pertinente en cada instante del procesamiento, emulando, en cierto modo, la atención selectiva humana.

3.0.2.1 *Matrices de Consulta, Clave y Valor (Query, Key, Value)*

La orquestación de la atención escalonada se basa en la interacción estratégica de tres matrices fundamentales: **Consulta (Query)**, **Clave (Key)** y **Valor (Value)**. Estas matrices, representadas convencionalmente como Q , K y V respectivamente, no son entidades independientes, sino que se derivan de la secuencia de entrada original a través de proyecciones lineales individuales y dedicadas. Para comprender mejor su función, podemos recurrir a una analogía conceptual:

- **Consulta (Q):** La matriz de Consulta (Q) personifica la pregunta o la necesidad de información del elemento que se está procesando en un momento dado. Cada fila de Q representa un elemento de la secuencia de entrada y encapsula la consulta.^o la búsqueda de información relevante en el resto de la secuencia que emana de dicho elemento. Imaginemos que cada palabra en una frase es una consulta "buscando entender su contexto.
- **Clave (K):** La matriz de Clave (K) actúa como un catálogo de claves.^o etiquetas descriptivas que identifican las diferentes partes de la secuencia de entrada. Al igual que Q , cada fila de K corresponde a un elemento de la secuencia de entrada, pero en este caso, representa la clave "que describe la información contenida en ese elemento y que podría ser relevante para responder a las consultas. Siguiendo con la analogía, cada palabra en la frase también proporciona una clave "que describe su propio significado y rol en la oración.
- **Valor (V):** La matriz de Valor (V) almacena la información concreta o el "valor.^a asociado a cada clave". De nuevo, cada fila de V se corresponde con un elemento de la secuencia de entrada y contiene la información que realmente se utilizará para construir la salida final, una vez ponderada por la atención. En nuestra analogía, la matriz de "valor" contendría la representación vectorial del significado de cada palabra, lista para ser combinada y ponderada para formar una representación contextualizada de la frase.

La esencia matemática del mecanismo de atención escalonada se encapsula en la Ecuación 1:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \quad (4)$$

donde:

- $Q \in \mathbb{R}^{n \times d_k}$ representa la matriz de Consulta, con n siendo la longitud de la secuencia y d_k la dimensión de las consultas y claves.
- $K \in \mathbb{R}^{m \times d_k}$ es la matriz de Clave, con m siendo la longitud de la secuencia (que puede ser diferente de n en algunos casos, como la atención Encoder-Decoder) y d_k la dimensión de las claves.
- $V \in \mathbb{R}^{m \times d_v}$ denota la matriz de Valor, con m siendo la longitud de la secuencia de valores y d_v la dimensión de los valores.
- d_k es el factor de escala, específicamente la raíz cuadrada de la dimensión de las claves (d_k), que se utiliza para mitigar el problema del desvanecimiento del gradiente, especialmente con dimensiones elevadas.
- softmax es la función softmax, encargada de normalizar las puntuaciones de similitud, transformándolas en una distribución de probabilidad que representa los pesos de atención.

El proceso computacional que subyace a la atención escalonada se desarrolla en los siguientes pasos:

1. **Cálculo de Similitud (Producto Punto):** Inicialmente, se evalúa la similitud entre cada consulta y cada clave mediante el **producto punto matricial** QK^T . Este producto punto cuantifica el grado de correspondencia o alineamiento entre las consultas y las claves, generando una matriz de **puntuaciones de similitud**.
2. **Escalado:** Las puntuaciones de similitud obtenidas se someten a un proceso de **escalado**, dividiéndolas por $\sqrt{d_k}$. Este paso de escalado es de vital importancia para la **estabilidad del entrenamiento**, ya que previene que las puntuaciones se disparen a valores excesivamente grandes, lo cual podría conducir a **gradientes extremadamente pequeños** tras la aplicación de la función softmax, dificultando el **aprendizaje efectivo**.
3. **Normalización (Softmax):** A continuación, la **función softmax** entra en juego para **normalizar** las puntuaciones escaladas. La softmax transforma estas puntuaciones en **pesos de atención probabilísticos**, que oscilan entre 0 y 1 y suman 1 para cada consulta. Estos **pesos de atención** reflejan la **importancia relativa** de cada valor con respecto a cada consulta, indicando en qué medida cada valor debe contribuir a la representación final.
4. **Ponderación y Agregación (Ponderación por Valores):** Finalmente, los pesos de atención normalizados se utilizan para **ponderar** la matriz de Valor V . Esta ponderación se materializa multiplicando los pesos de atención por los valores correspondientes y agregando los resultados mediante una **suma ponderada**. La **salida resultante** de este proceso es una matriz que representa la secuencia de entrada, pero ahora **enriquecida con información contextualizada**, donde cada

elemento se ha refinado y ajustado en función de su relevancia con respecto a los demás elementos de la secuencia, guiado por el mecanismo de atención.

3.0.3 Estructura de Capas en un Transformer

Un Transformer típico se compone de varias capas de codificación (**Encoder**) y decodificación (**Decoder**). Estas capas se apilan secuencialmente, creando una arquitectura profunda capaz de aprender representaciones complejas de los datos. Cada capa, tanto en el Encoder como en el Decoder, está construida con un conjunto específico de subcapas interconectadas, diseñadas para extraer información relevante y transformar los datos de entrada. Las subcapas fundamentales incluyen la **atención multi-cabeza** (Multi-Head Attention), **redes neuronales feed-forward**, **normalización de capas** (Layer Normalization) y **conexiones residuales**. A continuación, se detalla la estructura y función de cada componente.

3.0.3.1 Capas del Encoder

Cada capa del Encoder está diseñada para procesar la secuencia de entrada y extraer características relevantes. Típicamente, una capa de Encoder consta de las siguientes subcapas principales:

1. **Atención Multi-Cabeza (Multi-Head Attention):** Esta es la subcapa central del Transformer. Permite al modelo ponderar la importancia de diferentes partes de la secuencia de entrada al procesar cada posición. En lugar de una única atención, la atención multi-cabeza utiliza múltiples "cabezas" de atención en paralelo. Cada cabeza aprende a atender a diferentes aspectos de la información, capturando una variedad más rica de relaciones y dependencias dentro de la secuencia. La salida de todas las cabezas se concatena y se proyecta linealmente para producir la salida final de esta subcapa.
2. **Añadir y Normalizar (Add & Norm) - Conexión Residual y Normalización de Capas:** Después de la subcapa de atención multi-cabeza, se aplica una **conexión residual**. Esto significa que la entrada original a la subcapa de atención se suma a la salida de la atención multi-cabeza. Esta conexión residual ayuda a mitigar el problema del desvanecimiento del gradiente en redes profundas, facilitando el entrenamiento de modelos más profundos. A continuación, se aplica la **normalización de capas** (Layer Normalization) [?]. La normalización de capas ayuda a estabilizar el aprendizaje normalizando las salidas de las subcapas, lo que acelera el entrenamiento y mejora la generalización.
3. **Red Neuronal Feed-Forward (Feed-Forward Network):** Tras la subcapa de atención y la normalización, la salida se pasa a través de una **red neuronal feed-forward**. Esta red es aplicada de forma idéntica e independiente a cada posición en la secuencia. Consiste típicamente en dos capas lineales con una función de

activación no lineal (como ReLU) entre ellas. La red feed-forward ayuda a procesar la información aprendida por la atención, proporcionando no linealidad y permitiendo al modelo aprender transformaciones más complejas.

4. **Añadir y Normalizar (Add & Norm) - Conexión Residual y Normalización de Capas:** Similar al paso anterior, se aplica otra conexión residual y normalización de capas después de la red neuronal feed-forward. La entrada a la red feed-forward se suma a su salida, y el resultado se normaliza.

3.0.3.2 Capas del Decoder

Las capas del Decoder tienen la función de generar la secuencia de salida, utilizando la información codificada por el Encoder. Una capa de Decoder es similar a una capa de Encoder, pero incluye una subcapa de atención adicional para interactuar con la salida del Encoder. Las subcapas típicas de una capa de Decoder son:

1. **Atención Multi-Cabeza Enmascarada (Masked Multi-Head Attention):** Similar a la atención multi-cabeza del Encoder, pero con una modificación crucial: el **enmascaramiento**. En el contexto del Decoder, especialmente en tareas de generación secuencial, es importante que al predecir la salida en una posición dada, el modelo solo pueda atender a las posiciones *anteriores* en la secuencia de salida (ya generada) y no a las posiciones futuras. El enmascaramiento se implementa para evitar que el modelo "vea" la respuesta durante el entrenamiento, asegurando que aprenda a generar la secuencia de forma autoregresiva.
2. **Añadir y Normalizar (Add & Norm):** Conexión residual y normalización de capas aplicada después de la atención multi-cabeza enmascarada, de la misma manera que en el Encoder.
3. **Atención Encoder-Decoder (Encoder-Decoder Attention o Multi-Head Attention):** Esta subcapa es específica del Decoder y es fundamental para la interacción entre el Encoder y el Decoder. En esta subcapa, el Decoder atiende a la salida del *Encoder*. Las "queries" provienen de la salida de la subcapa de atención enmascarada del Decoder, mientras que las "keys" "values" provienen de la salida del *Encoder*. Esto permite al Decoder enfocar su atención en las partes relevantes de la secuencia de entrada codificada por el Encoder al generar cada elemento de la secuencia de salida.
4. **Añadir y Normalizar (Add & Norm):** Conexión residual y normalización de capas aplicada después de la atención Encoder-Decoder.
5. **Red Neuronal Feed-Forward (Feed-Forward Network):** Idéntica a la red neuronal feed-forward utilizada en las capas del Encoder.
6. **Añadir y Normalizar (Add & Norm):** Conexión residual y normalización de capas aplicada después de la red neuronal feed-forward.

3.0.3.3 Diagrama Conceptual de un Bloque Transformer

Un diagrama de un bloque Transformer típico mostraría:

- **Para un bloque Encoder:**
 - Un bloque etiquetado *.Entrada* que representa la secuencia de entrada (o la salida de la capa Encoder anterior).
 - Una flecha que lleva a un bloque etiquetado "Multi-Head Attention".
 - Una conexión residual que salta la subcapa "Multi-Head Attention" se suma a la salida de esta subcapa.
 - Un bloque etiquetado *.Add & Norm* después de la suma de la conexión residual.
 - Una flecha desde *.Add & Norm* a un bloque etiquetado "Feed-Forward Network".
 - Otra conexión residual que salta la subcapa "Feed-Forward Network" se suma a la salida de esta subcapa.
 - Un bloque etiquetado *.Add & Norm* después de la segunda suma de la conexión residual, representando la salida de la capa Encoder.
- **Para un bloque Decoder:**
 - Un bloque etiquetado *.Entrada* que representa la secuencia de entrada al Decoder (o la salida de la capa Decoder anterior).
 - Una flecha que lleva a un bloque etiquetado "Masked Multi-Head Attention".
 - Una conexión residual y *.Add & Norm* después de la atención enmascarada.
 - Una flecha a un bloque etiquetado *.Encoder-Decoder Attention* (o simplemente "Multi-Head Attention" si se entiende el contexto). Este bloque también recibiría una conexión de la salida del Encoder (en un diagrama para la arquitectura completa).
 - Una conexión residual y *.Add & Norm* después de la atención Encoder-Decoder.
 - Una flecha a un bloque etiquetado "Feed-Forward Network".
 - Una conexión residual y *.Add & Norm* después de la red feed-forward, representando la salida de la capa Decoder.

En resumen, la estructura de capas de un Transformer, con sus componentes de atención multi-cabeza, redes feed-forward, normalización de capas y conexiones residuales, proporciona un marco robusto y eficiente para el modelado de secuencias complejas, tanto en tareas de codificación como de decodificación. La atención multi-cabeza es el mecanismo central que permite al modelo capturar dependencias ricas, mientras que las otras subcapas facilitan el entrenamiento y mejoran la capacidad de aprendizaje de la red.

3.0.4 Retropropagación en Modelos Transformer

El entrenamiento de los modelos Transformer se realiza mediante el algoritmo de **retropropagación** (Backpropagation) [?], que ajusta los pesos del modelo iterativamente para minimizar una función de pérdida. En esencia, la retropropagación es un método para calcular eficientemente el gradiente de la función de pérdida con respecto a los parámetros del modelo (pesos y biases). Este gradiente indica la dirección en la que se deben ajustar los parámetros para reducir la pérdida y, por lo tanto, mejorar el rendimiento del modelo.

El proceso de retropropagación en un Transformer sigue los principios generales de este algoritmo, pero con consideraciones específicas para las capas de atención y las redes feed-forward que componen su arquitectura. A grandes rasgos, el proceso se puede describir en los siguientes pasos:

1. **Paso Forward (Paso hacia adelante):** Se propaga la secuencia de entrada a través de la red Transformer, desde la capa de entrada hasta la capa de salida. En este paso, se calculan las salidas de cada subcapa (atención multi-cabeza, redes feed-forward, normalización, etc.) secuencialmente. Específicamente, en la subcapa de atención, se calculan las matrices de Consulta (Q), Clave (K) y Valor (V), se realiza el producto punto escalado, se aplica la función Softmax para obtener los pesos de atención y finalmente se calcula la salida ponderada por los valores. Las redes feed-forward aplican transformaciones lineales y no lineales a la salida de la atención.
2. **Cálculo de la Pérdida:** Una vez obtenida la salida del modelo para la secuencia de entrada, se compara con la salida deseada (la "verdad ground") utilizando una función de pérdida adecuada para la tarea en cuestión (e.g., cross-entropy para clasificación, error cuadrático medio para regresión). Esta función de pérdida cuantifica la discrepancia entre la predicción del modelo y el valor real.
3. **Paso Backward (Paso hacia atrás):** Se inicia el proceso de retropropagación propiamente dicho. El gradiente de la función de pérdida se calcula con respecto a la salida del modelo. Luego, utilizando la **regla de la cadena del cálculo diferencial**, este gradiente se propaga hacia atrás a través de cada capa de la red, desde la capa de salida hasta la capa de entrada.
 - **Retropropagación a través de la Atención Multi-Cabeza:** La retropropagación a través de la atención es un poco más compleja que en las capas lineales tradicionales. El gradiente se propaga a través de la función Softmax, el escalado, el producto punto, y de vuelta a través de las matrices de Consulta, Clave y Valor. Esto implica calcular los gradientes con respecto a los pesos de proyección que generaron Q, K y V, así como con respecto a las entradas a la capa de atención. Es importante tener en cuenta que el gradiente también se propaga a través de los pesos de atención, afectando

a todas las partes de la secuencia de entrada que contribuyeron a la salida actual.

- **Retropropagación a través de Redes Feed-Forward:** La retropropagación a través de las redes feed-forward es similar a la de las redes neuronales multicapa estándar. El gradiente se propaga a través de la función de activación no lineal (e.g., ReLU) y las capas lineales, calculando los gradientes con respecto a los pesos y biases de estas capas.
 - **Retropropagación a través de Add & Norm y Conexiones Residuales:** La retropropagación a través de las **conexiones residuales** es sencilla: el gradiente se divide y se suma tanto al camino principal como al camino de la conexión residual. En las capas de normalización, la retropropagación implica calcular los gradientes a través de las operaciones de normalización (media, varianza) y los parámetros aprendibles de la normalización (si los hay, como en Layer Normalization con parámetros gamma y beta).
4. **Actualización de Pesos:** Una vez calculados los gradientes para todos los parámetros del modelo, se utilizan para actualizar los pesos y biases en la dirección opuesta al gradiente, utilizando un algoritmo de optimización como el descenso de gradiente (Gradient Descent) o sus variantes (e.g., Adam, SGD con momentum). La tasa de aprendizaje (learning rate) controla el tamaño del paso en la dirección del gradiente.

3.0.4.1 Consideraciones Específicas para Transformers en Retropropagación

- **Costo Computacional de la Atención:** El cálculo de la atención, especialmente en secuencias largas, puede ser computacionalmente costoso tanto en el paso forward como en el backward. La complejidad de la atención escalonada es cuadrática con respecto a la longitud de la secuencia ($O(L^2)$, donde L es la longitud de la secuencia). Esto puede ser un cuello de botella durante el entrenamiento, especialmente para secuencias muy largas. Técnicas como la atención dispersa (sparse attention) o la atención lineal se han propuesto para mitigar este problema.
- **Estabilidad del Gradiente:** Las conexiones residuales y la normalización de capas en los Transformers ayudan a mejorar la estabilidad del gradiente durante la retropropagación, facilitando el entrenamiento de redes profundas y reduciendo problemas como el desvanecimiento o la explosión del gradiente.
- **Paralelización:** Aunque el mecanismo de atención permite la paralelización en el paso forward, la retropropagación también se puede paralelizar en gran medida, especialmente en hardware como GPUs, lo que acelera significativamente el entrenamiento de los Transformers.

En resumen, la retropropagación en los modelos Transformer es una aplicación del algoritmo estándar, adaptada a la estructura específica de la red, con especial atención al cálculo de gradientes a través del mecanismo de atención. La eficiencia y

estabilidad del entrenamiento se ven favorecidas por las características arquitectónicas de los Transformers, como la atención multi-cabeza, las conexiones residuales y la normalización de capas.

3.1 SELF-ATTENTION-BASED IMPUTATION FOR TIME SERIES (SAITS)

SAITS (Self-Attention-based Imputation for Time Series) es un modelo Transformer específicamente diseñado para la imputación de datos faltantes en series temporales. Aprovecha el mecanismo de auto-atención para capturar dependencias temporales y espaciales en los datos de series temporales, lo que permite una imputación más precisa en comparación con los métodos tradicionales. A diferencia de los modelos de imputación clásicos que a menudo se basan en suposiciones simplificadas sobre la estructura temporal de los datos, SAITS aprende directamente estas dependencias de los propios datos, ofreciendo una solución más flexible y robusta.

3.1.1 *Arquitectura del Modelo SAITS*

SAITS adapta la arquitectura Transformer estándar para la tarea de imputación de series temporales, principalmente utilizando una estructura de **Encoder Transformer**. Dado que el objetivo principal es la imputación y no la generación secuencial, la arquitectura Decoder no es estrictamente necesaria en este contexto, aunque variaciones podrían incluirla. Las adaptaciones clave en la arquitectura de SAITS para la imputación de series temporales son:

- **Entrada de Series Temporales:** SAITS toma como entrada series temporales con posibles valores faltantes. Típicamente, la serie temporal se representa como una secuencia de vectores, donde cada vector representa un punto de tiempo y contiene las mediciones de diferentes variables en ese instante. Para manejar los valores faltantes, se pueden emplear **máscaras de entrada** que indican las posiciones donde los datos son desconocidos. Estas máscaras pueden ser utilizadas por el modelo para enfocar la atención en las partes observadas de la serie temporal durante la imputación.
- **Encoder Transformer para Imputación:** SAITS utiliza un stack de capas Encoder Transformer. Cada capa Encoder, como se describió anteriormente, contiene subcapas de atención multi-cabeza, redes feed-forward, normalización de capas y conexiones residuales. Estas capas procesan la serie temporal de entrada para aprender representaciones latentes que capturan las dependencias temporales y las relaciones entre las variables. La **auto-atención** dentro del Encoder permite que cada punto de tiempo en la serie temporal atienda a todos los demás puntos, ponderando su importancia relativa para la imputación en la posición actual.

- **Codificación Posicional Adaptada (Opcional):** Si bien la codificación posicional es crucial en NLP para indicar el orden secuencial de las palabras, su uso en series temporales puede ser adaptado. En algunos casos, la posición temporal relativa ya está implícita en la secuencia de entrada. Sin embargo, se pueden utilizar codificaciones posicionales para explicitar la posición temporal y potencialmente codificar información adicional como el tiempo transcurrido desde el inicio de la serie o patrones temporales cíclicos (e.g., día de la semana, mes del año). La decisión de incluir o adaptar la codificación posicional puede depender de las características específicas del conjunto de datos de series temporales.
- **Capa de Salida para Imputación:** La salida del Encoder Transformer, tras pasar por las capas Encoder, se utiliza para generar los valores imputados. Típicamente, esto se realiza mediante una **capa lineal** final que proyecta la representación latente de cada punto de tiempo a la dimensión del espacio de características original. Esta capa lineal aprende a mapear las representaciones aprendidas por el Transformer a los valores imputados para cada variable en cada punto de tiempo donde falta información.

3.1.2 Función de Pérdida para SAITS

La función de pérdida comúnmente utilizada para entrenar SAITS en tareas de imputación es el **Error Cuadrático Medio (MSE)** o variantes del mismo, aplicadas **solo a las posiciones donde los datos estaban originalmente faltantes**. Esto se debe a que el objetivo es minimizar el error en la predicción de los valores faltantes, sin penalizar al modelo por los valores ya observados. Formalmente, si Y es la serie temporal original (con valores faltantes), $\hat{Y}$ es la serie temporal imputada por SAITS y M es una matriz máscara binaria donde $M_{ij} = 1$ si el valor Y_{ij} estaba faltante y $M_{ij} = 0$ si estaba observado, la función de pérdida MSE puede definirse como:

$$\mathcal{L}_{MSE} = \frac{1}{\sum_{i,j} M_{ij}} \sum_{i,j} M_{ij} (Y_{ij} - \hat{Y}_{ij})^2 \quad (5)$$

Esta función de pérdida asegura que el modelo se centre en aprender a imputar los valores faltantes con precisión, ignorando el error en las posiciones donde ya se disponía de información. Otras funciones de pérdida robustas, como el Mean Absolute Error (MAE) o funciones de pérdida basadas en cuantiles, también podrían ser consideradas dependiendo de las características de los datos y los objetivos específicos de la imputación.

3.1.3 *Ventajas de Self-Attention para Imputación de Series Temporales*

El uso del mecanismo de auto-atención en SAITS ofrece varias ventajas significativas para la imputación de series temporales en comparación con métodos tradicionales:

- **Modelado de Dependencias a Largo Plazo:** La auto-atención permite a SAITS capturar dependencias temporales a largo plazo de manera efectiva. A diferencia de métodos como ARIMA o RNNs que pueden tener dificultades para recordar información a través de largos periodos de tiempo, la atención permite a SAITS acceder directamente a cualquier punto en la serie temporal al imputar un valor, sin importar la distancia temporal. Esto es crucial para series temporales con patrones complejos y dependencias que se extienden a lo largo del tiempo.
- **Captura de Dependencias Espaciales y Temporales Conjuntas:** En series temporales multivariadas, donde se miden múltiples variables a lo largo del tiempo, la auto-atención puede capturar tanto las dependencias temporales dentro de cada variable como las dependencias espaciales (correlaciones) entre diferentes variables en diferentes puntos de tiempo. El modelo aprende a imputar un valor faltante considerando la información de todas las variables en todos los puntos de tiempo relevantes.
- **Flexibilidad y Adaptabilidad:** La arquitectura Transformer es inherentemente flexible y adaptable a diferentes tipos de series temporales y patrones de datos faltantes. SAITS no requiere suposiciones fuertes sobre la estacionariedad de la serie temporal o la naturaleza de las dependencias temporales. Puede aprender patrones complejos directamente de los datos y adaptarse a diferentes tipos de series temporales sin necesidad de ingeniería de características manual extensiva.
- **Robustez ante Datos Irregulares y Valores Atípicos:** La atención puede ayudar a SAITS a ser más robusto ante datos irregulares y valores atípicos en las series temporales. Al ponderar la importancia de diferentes partes de la secuencia, la atención puede dar menos peso a los valores atípicos o a las regiones ruidosas de la serie temporal durante la imputación, enfocándose en la información más consistente y relevante.

En resumen, SAITS representa un avance significativo en la imputación de series temporales al aplicar la poderosa arquitectura Transformer y el mecanismo de auto-atención. Su capacidad para modelar dependencias complejas, su flexibilidad y robustez lo convierten en una herramienta valiosa para el análisis de datos de series temporales con valores faltantes en diversos dominios.

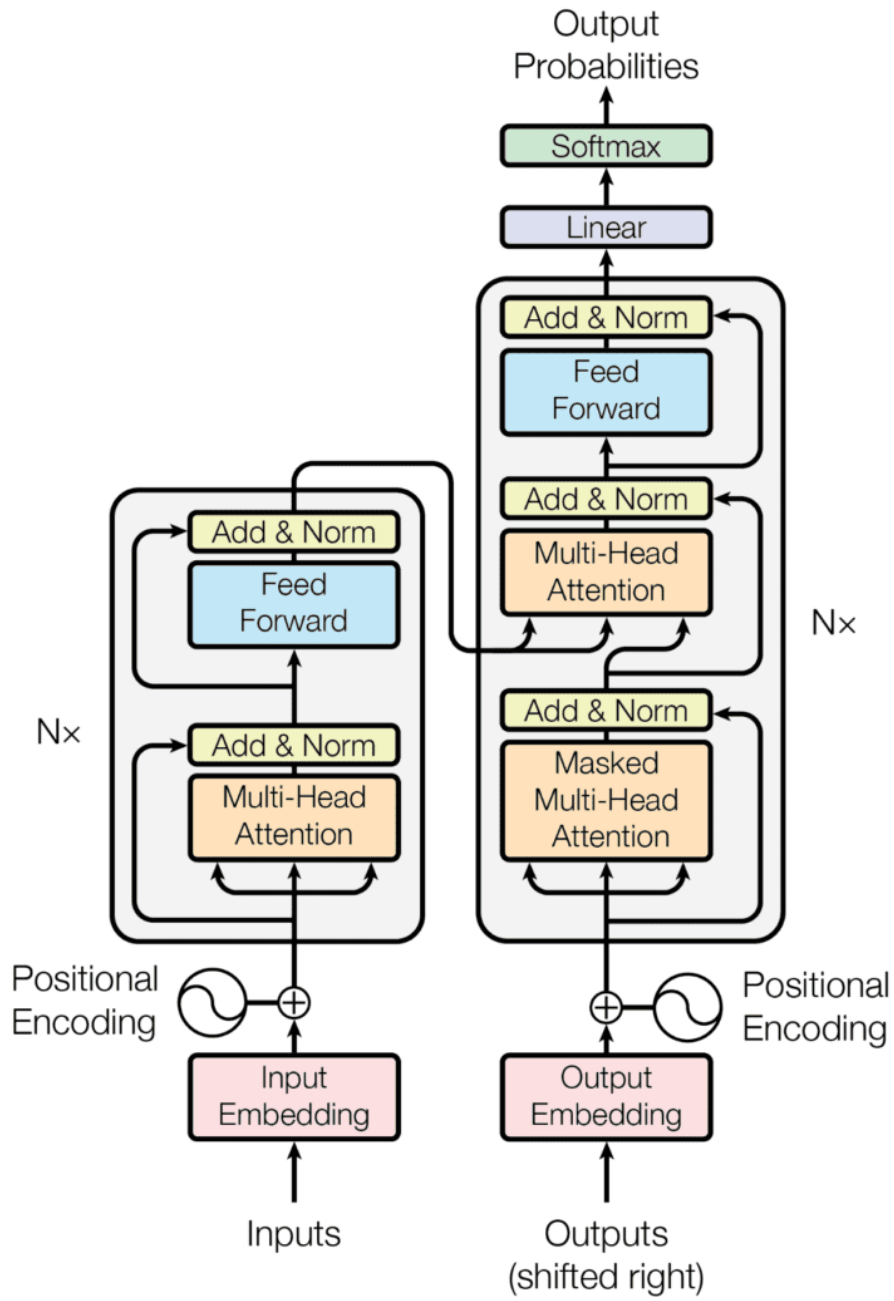


Figura 1: Diagrama conceptual de un bloque Transformer típico (Encoder y Decoder).

IMPUTACIÓN DE DATOS DE TRÁFICO Y CONTAMINACIÓN

4.1 CONTEXTO DEL PROBLEMA Y MOTIVACIÓN

El presente estudio se enmarca en el ámbito de las *Smart Villages* o pueblos inteligentes, aplicando tecnologías del Internet de las Cosas (IoT) para monitorizar y analizar dinámicas ambientales y de movilidad en entornos rurales. Específicamente, se utiliza la infraestructura desplegada en el proyecto Smart Poqueira [?], localizado en la Alpujarra Granadina, una zona de alto valor ecológico y turístico. La infraestructura IoT consta, entre otros sensores, de cámaras con reconocimiento de matrículas (LPR - License Plate Recognition) estratégicamente ubicadas en los accesos a los municipios de Pampaneira, Bubión y Capileira (Figura ??), y una unidad móvil (camión) equipada con sensores para medir diversos contaminantes atmosféricos y variables meteorológicas.

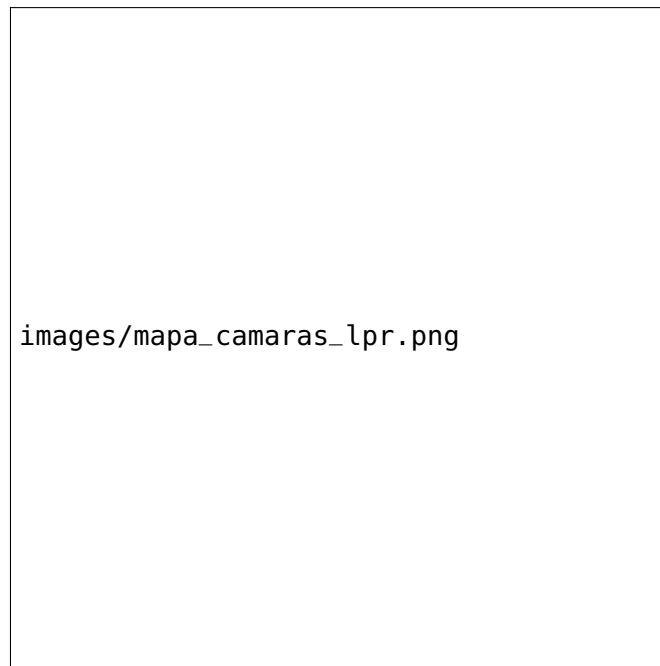


Figura 2: Distribución geográfica de las cámaras LPR (License Plate Recognition) en los municipios de Pampaneira, Capileira y Bubión, utilizadas para la monitorización del tráfico vehicular en el proyecto Smart Poqueira. Se indican las ubicaciones específicas y la dirección de monitorización.

El objetivo general es comprender las relaciones entre los patrones de tráfico vehicular y los niveles de contaminación atmosférica en esta región. Sin embargo, surge un desafío fundamental inherente a la naturaleza móvil de la unidad de medición de contaminantes: el camión de meteorología y contaminación opera en ubicaciones específicas (Pampaneira-Bubión) solo durante periodos de tiempo definidos:

- **Capileira:** 16/11/2022 - 15/01/2023 (Estos datos no se tratan en este análisis específico de imputación).
- **Pampaneira-Bubión (Cementerio):** 17/01/2023 - 14/03/2023.
- **Pampaneira-Bubión (Cementerio):** 06/06/2023 - 27/06/2023.

Esta operativa, junto con posibles fallos intermitentes de los sensores o cámaras, genera **datos faltantes** sistemáticos en las series temporales combinadas de tráfico y contaminación. La ausencia de estas mediciones impide un análisis continuo y completo de la relación tráfico-contaminación.

Por lo tanto, la **motivación principal** de este trabajo es abordar el problema de los datos faltantes mediante la aplicación y evaluación de diferentes técnicas de **imputación**. El objetivo es estimar los valores ausentes de manera robusta y precisa, aprovechando la información contenida en los datos existentes, para así poder realizar análisis posteriores más completos y fiables.

4.2 CONJUNTOS DE DATOS

Para llevar a cabo este estudio, se utilizan dos conjuntos de datos principales, derivados de la infraestructura IoT descrita.

4.2.1 *Dataset de Tráfico General*

Este dataset contiene los registros agregados por hora del tráfico vehicular detectado por las 4 cámaras LPR durante un periodo extenso (febrero 2022 - agosto 2023). Cada fila representa una hora y las columnas cuantifican vehículos categorizados por: localización/dirección (PAM_1, PAM_2, BUB, CAP; IN/OUT), origen geográfico (Zona_Granada, Zona_Andalucia_no_GR, etc.), y características inferidas (asientos, CO₂, etiqueta DGT). La columna nan agrupa vehículos sin información específica. Proporciona una visión continua de la movilidad.

4.2.2 *Dataset de Intersección Tráfico-Contaminación*

Este dataset fusiona temporalmente los datos de tráfico con las mediciones del camión móvil, conteniendo únicamente registros horarios de los periodos y ubicaciones donde el camión estuvo operativo en Pampaneira/Bubión. Es un subconjunto temporal del dataset de tráfico, enriquecido con variables ambientales.

4.2.2.1 *Variables de Contaminación*

- co Monóxido de Carbono:** Concentración media horaria medida con el analizador Thermo Scientific mod. 48i. Es un trazador de combustión incompleta, principalmente de vehículos de gasolina en condiciones de arranque en frío o ralentí, y también de la quema de biomasa. Unidades: miligramos por metro cúbico (mg/m³).
- no Monóxido de Nitrógeno:** Concentración media horaria medida con el analizador Thermo Scientific mod. 42i. Es un contaminante primario emitido directamente por procesos de combustión a alta temperatura, siendo un **trazador importante del tráfico diésel**. Unidades: microgramos por metro cúbico (µg/m³).
- no2 Dióxido de Nitrógeno:** Concentración media horaria medida con el analizador Thermo Scientific mod. 42i. Se forma principalmente por la oxidación del NO en la atmósfera, aunque también se emite directamente en menor medida. Es, junto con el NO, un **trazador clave del tráfico diésel** y otros procesos de combustión. Unidades: microgramos por metro cúbico (µg/m³).

- o3 Ozono Troposférico:** Concentración media horaria medida con el analizador Thermo Scientific mod. 49i. Es un contaminante secundario, no emitido directamente, sino formado en la atmósfera por reacciones fotoquímicas entre óxidos de nitrógeno (NOx) y compuestos orgánicos volátiles (COVs) en presencia de luz solar. Unidades: microgramos por metro cúbico ($\mu\text{g}/\text{m}^3$).
- s02 Dióxido de Azufre:** Concentración media horaria medida con el analizador Thermo Scientific mod. 43i. Proviene principalmente de la combustión de combustibles fósiles que contienen azufre (carbón, fuelóleo). Su relevancia como trazador del tráfico rodado actual es limitada debido a las bajas concentraciones de azufre en gasolinas y gasóleos modernos. Unidades: microgramos por metro cúbico ($\mu\text{g}/\text{m}^3$).
- pm10 Material Particulado <10 μm :** Concentración media horaria de partículas en suspensión con un diámetro aerodinámico inferior a 10 micrómetros, obtenida con el Monitor TEOM 1405 (Thermo Scientific). Incluye una mezcla compleja de partículas de diversas fuentes: combustión (tráfico, industria, calefacción), polvo resuspendido, aerosoles secundarios, fuentes naturales, etc. Unidades: microgramos por metro cúbico ($\mu\text{g}/\text{m}^3$).
- ebc_tot Carbono Negro Total (Hollín):** Concentración media horaria de carbono negro total obtenida con el fotómetro de absorción Aethalometer AE31. El carbono negro es un producto de la combustión incompleta de material carbonoso y es un componente importante del material particulado fino. Unidades: microgramos por metro cúbico ($\mu\text{g}/\text{m}^3$).
- ebc_ff Carbono Negro de Combustibles Fósiles:** Concentración media horaria de carbono negro atribuible específicamente a la combustión de combustibles fósiles (gasolina, diésel). Se obtiene aplicando el "modelo del etalómetro"[?]. Es considerado un **trazador primario y específico del tráfico vehicular**. Unidades: microgramos por metro cúbico ($\mu\text{g}/\text{m}^3$).
- ebc_bb Carbono Negro de Quema de Biomasa:** Concentración media horaria de carbono negro atribuible específicamente a la quema de biomasa (leña para calefacción, quemas agrícolas, incendios forestales). Se obtiene aplicando el "modelo del etalómetro"[?]. Unidades: microgramos por metro cúbico ($\mu\text{g}/\text{m}^3$).

4.2.2.2 Variables Meteorológicas

- temp Temperatura del Aire:** Temperatura media horaria medida por el transmisor meteorológico YOUNG 92000 ResponseONE. Influye en la tasa de reacciones químicas atmosféricas y en la dispersión vertical de los contaminantes. Temperaturas altas en invierno pueden indicar condiciones anticiclónicas de estancamiento. Unidades: grados Celsius ($^{\circ}\text{C}$).

- rh Humedad Relativa:** Humedad relativa media horaria del aire medida por el transmisor YOUNG 92000. Afecta a la formación de aerosoles secundarios, la visibilidad y puede influir en la deposición de contaminantes. Alta humedad puede correlacionarse con condiciones de estancamiento o niebla. Unidades: porcentaje (%).
- ws Velocidad del Viento:** Velocidad media horaria del viento medida por el anemómetro de la torre meteorológica. Es un factor fundamental para la dispersión horizontal de los contaminantes; velocidades bajas favorecen la acumulación. Unidades: metros por segundo (m/s).
- wd Dirección del Viento:** Dirección media horaria desde la que sopla el viento, medida en la torre meteorológica. 0° corresponde al Norte geográfico. Es crucial para identificar el transporte de contaminantes desde posibles áreas fuente. Unidades: grados (°).
- pres Presión Atmosférica:** Presión atmosférica media horaria a nivel de la estación de medición. Altas presiones suelen asociarse con condiciones meteorológicas estables (anticiclones) que, combinadas con vientos bajos, dificultan la dispersión vertical y horizontal de los contaminantes, favoreciendo episodios de alta contaminación. Unidades: hectopascales (hPa).

4.2.2.3 *Variable de Posición del Camión*

truck_pos Posición del Camión Meteorológico: Variable categórica que indica la cámara LPR más cercana a la ubicación física del camión de medición durante la hora correspondiente. Los valores posibles en este dataset son 'CAP' (cuando estaba en Capileira, aunque esos datos no se usan aquí para la imputación) y 'PAM_2' (cuando estaba en la ubicación del cementerio de Pampaneira, monitorizado por la cámara PAM_2). Esta variable permite asociar las mediciones de contaminación con el flujo de tráfico específico de la cámara más próxima.

Este dataset hereda también las columnas de tráfico vehicular correspondientes a la hora y cámara más cercana. Es el dataset principal para el análisis y la imputación.

4.3 ANÁLISIS EXPLORATORIO DE DATOS (EDA)

Se realizó un Análisis Exploratorio de Datos (EDA) sobre el dataset de trafico-intersección para comprender las distribuciones, identificar patrones, valores atípicos, datos faltantes y correlaciones.



Figura 3: Distribución del número de registros horarios en el dataset de intersección (trafico_contamina_interseccion) según la ubicación y periodo operativo del camión móvil de medición (PAM Periodo 1: Ene-Mar 2023, PAM Periodo 2: Jun 2023).

4.3.1 *Análisis de Valores Faltantes*

El análisis del porcentaje de valores faltantes por variable (Figura ??) muestra una alta incidencia de datos ausentes en varias columnas clave. Destaca eBC_tot con casi el 70 % de sus valores faltantes, seguido por eBC y SO₂ con cerca del 50 %. Un segundo grupo de variables afectadas significativamente incluye WS, WD (ambas 26.6 %), PM₁₀ (20.3 %) y las fracciones eBC_ff y eBC_bb (ambas 18.8 %). La considerable proporción de datos faltantes, particularmente en indicadores importantes de calidad del aire y fuentes como eBC_tot, SO₂ y PM₁₀, confirma la necesidad de realizar una imputación robusta para análisis posteriores.



Figura 4: Porcentaje de valores faltantes (nulos) para las 15 variables con mayor proporción de ausencias en el dataset de intersección trafico_contamina_interseccion. Se observa la mayor incidencia en eBC_tot (69.3 %), seguida por eBC (49.5 %) y SO₂ (48.8 %). Variables como WS, WD, y PM₁₀ también presentan una notable proporción de datos ausentes superior al 20 %.

4.3.2 Distribución de Variables

Se analizaron las distribuciones mediante histogramas y diagramas de caja.

- **Contaminantes (CO, NO, NO₂):** Los histogramas (Figura ??) mostraron distribuciones con fuerte asimetría derecha, indicando predominio de concentraciones bajas con picos ocasionales altos, característico de fuentes como el tráfico.

images/EDA/histogramas_co.png

Figura 5: Histogramas de las concentraciones horarias de CO, NO y NO₂. Se observa una clara asimetría hacia la derecha en las tres distribuciones.

- **Contaminantes (O₃, SO₂, PM₁₀, eBC):** Los diagramas de caja (Figura ?? y revelaron valores atípicos, especialmente en O₃ y PM₁₀. La mediana de SO₂

fue muy baja. Las fracciones de eBC mostraron diferencias en sus rangos y medianas.



Figura 6: Diagramas de caja comparativos para variables de contaminación. A la izquierda se muestran O₃, SO₂ y PM₁₀. A la derecha se muestran las variables relacionadas con el carbono negro elemental (eBC_{tot}, eBC_{ff}, eBC_{bb}).

- **Meteorología (TEMP, RH, WS):** Los histogramas (Figura ??) mostraron distribuciones más variadas: TEMP bimodal (ciclos día/noche), RH más centrada, y WS con fuerte asimetría derecha (predominio de vientos bajos).

images/EDA/histogramas_3.png

Figura 7: Histogramas de las variables meteorológicas TEMP (Temperatura), RH (Humedad Relativa) y WS (Velocidad del Viento).

4.3.3 *Análisis de Correlaciones*

La matriz de correlación (Figura ??) mostró relaciones significativas:

- **Positivas Fuertes:** eBC_tot con eBC_ff (0.76) y eBC_bb (0.64); NO₂ con eBC_tot (0.69); NO con NO₂ (0.64); O₃ con TEMP (0.4).
- **Negativas Moderadas:** RH con O₃ (-0.5); RH con TEMP (-0.5).
- **Otras:** Correlaciones negativas débiles entre contaminantes primarios (NO_x, eBC) y O₃. Correlaciones débiles entre meteorología y contaminantes primarios.

Estas correlaciones, especialmente las ligadas a trazadores de tráfico, son clave para la imputación basada en modelos predictivos.



Figura 8: Matriz de correlación de Pearson entre las principales variables de contaminación

4.4 METODOLOGÍA: PIPELINE DE IMPUTACIÓN Y FORMALIZACIÓN

Este capítulo describe la metodología completa empleada para la imputación de valores faltantes. Se detalla la preparación de los datos, la configuración experimental para evaluación, y la aplicación de cada técnica de imputación implementada dentro del módulo `pampaneira_imputation`.

4.5 PREPARACIÓN DE DATOS Y CONFIGURACIÓN EXPERIMENTAL

El proceso se inicia con los datos crudos de series temporales multivariantes de Pampaneira-Bubión.

4.5.1 Carga, Estructura y Preprocesamiento Inicial

Los datos originales, que pueden contener faltantes naturales, son cargados utilizando rutinas del módulo `pampaneira_imputation.data_loader`. Posteriormente, una función del módulo `pampaneira_imputation.data_preprocessor` asegura un índice temporal horario completo y regular para los periodos de interés definidos. Los datos se organizan en una estructura tensorial 3D ($N \times T \times D$), donde N es el número de muestras (ventanas temporales), $T = 24$ es la longitud fija de la ventana, y D el número de características. Los datos se dividen por periodos temporales para análisis separados.

4.5.2 División, Escalado y Creación del Escenario de Evaluación

El núcleo del preprocesamiento, gestionado por el módulo `pampaneira_imputation.data_preprocessor`, aplica los siguientes pasos a los datos de cada periodo:

1. **División Train/Validation/Test:** Segmentación cronológica de los datos en conjuntos de entrenamiento (80 %), validación (20 % del de entrenamiento) y prueba (20 % final), según las duraciones especificadas.
2. **Escalado (Normalización Z-score):** Aplicación de escalado estándar a las características seleccionadas, ajustando el escalador (*fitting*) solo sobre el conjunto de entrenamiento y aplicándolo (*transforming*) a los tres conjuntos.
3. **Ventaneo (Windowing):** Creación de secuencias de longitud $T = 24$ mediante una ventana deslizante, resultando en la estructura tensorial 3D ($N \times T \times D$).
4. **Introducción Artificial de Datos Faltantes:** Se eliminan artificialmente un 10 % de los valores previamente observados (después de escalar y ventanear) en cada split (train, val, test), siguiendo un patrón configurado (ej., 'point'). Este paso es crucial para la evaluación controlada.

Este proceso define formalmente los siguientes conjuntos de datos y máscaras utilizados en el experimento:

- $\mathcal{X}_{\text{orig}}$: El tensor de datos escalados y ventaneados *antes* de introducir los faltantes artificiales. Contiene los faltantes naturales originales y sirve como **ground truth** para la evaluación.
- $\mathbf{M}_{\text{natural}}$: La máscara binaria que indica la posición de los faltantes naturales en $\mathcal{X}_{\text{orig}}$ (0 =faltante, 1 =observado).
- $\mathbf{M}_{\text{artificial}}$: La máscara binaria que indica las posiciones donde se introdujeron faltantes *artificialmente* (1 =eliminado artificialmente, 0 =no eliminado).
- $\mathcal{X}_{\text{missing}}$: El tensor de datos utilizado como **entrada** para los métodos de imputación. Contiene tanto los faltantes naturales como los artificiales.

- **M**: La **máscara combinada** para $\mathcal{X}_{\text{missing}}$, indicando *todas* las posiciones faltantes (0 =faltante, 1 =observado).

$$\mathbf{M} = \mathbf{M}_{\text{natural}} \odot (\mathbf{1} - \mathbf{M}_{\text{artificial}})$$

4.6 PROBLEMA DE IMPUTACIÓN Y OBJETIVO DE EVALUACIÓN

Definición 27 (Problema de Imputación Experimental y Evaluación). El problema abordado es desarrollar y evaluar funciones de imputación f que, aplicadas al conjunto de test con faltantes combinados $(\mathcal{X}_{\text{missing}}^{\text{test}}, \mathbf{M}^{\text{test}})$, generen una estimación completada $\hat{\mathcal{X}}^{\text{test}}$:

$$\hat{\mathcal{X}}^{\text{test}} = f(\mathcal{X}_{\text{missing}}^{\text{test}}, \mathbf{M}^{\text{test}})$$

Para modelos predictivos f_{θ} , la función f depende de parámetros θ^* optimizados durante una fase de entrenamiento previa. El **objetivo principal de evaluación** es medir la precisión con la que $\hat{\mathcal{X}}^{\text{test}}$ reconstruye los valores verdaderos $\mathcal{X}_{\text{orig}}^{\text{test}}$ exclusivamente en las posiciones indicadas por la máscara artificial $\mathbf{M}_{\text{artificial}}^{\text{test}}$ (o $\mathbf{M}_{\text{indicating}}^{\text{test}}$):

$$\text{Error} = \text{Metric}(\hat{\mathcal{X}}^{\text{test}} \odot \mathbf{M}_{\text{artificial}}^{\text{test}}, \mathcal{X}_{\text{orig}}^{\text{test}} \odot \mathbf{M}_{\text{artificial}}^{\text{test}})$$

Se busca la función f que minimice este error (medido por MAE, MSE, RMSE, MRE) en el conjunto de test.

4.7 IMPLEMENTACIÓN DE LAS FUNCIONES DE IMPUTACIÓN

Las siguientes subsecciones describen la implementación conceptual de cada función de imputación f , gestionada por el módulo `pampaneira_imputation.imputation_methods`, aplicada sobre $\mathcal{X}_{\text{missing}}^{\text{test}}$.

4.7.1 Métodos Baseline o Estadísticos Simples

Imputación por Media/Mediana Sample-Wise

Procesa cada muestra $\mathbf{X}^{(k)}$ de 24 horas independientemente. Para cada característica, calcula la media/mediana usando solo los valores observados dentro de esa muestra y rellena los NaN de esa característica en esa muestra con el valor calculado. Implementa una lógica de fallback (usar estadística global del training set o 0) si una característica es completamente NaN dentro de una muestra.

4.7.2 Métodos Basados en Propagación e Interpolación

Estos métodos también operan independientemente sobre cada muestra $\mathbf{X}^{(k)}$ utilizando técnicas estándar de series temporales (implementadas internamente con ayuda de bibliotecas como pandas).

Imputación Hacia Adelante (Forward Fill / LOCF)

Propaga la última observación válida hacia adelante en el tiempo ($\text{axis}=0$). Los NaN iniciales o persistentes se rellenan con 0.

Imputación Hacia Atrás (Backward Fill / NOCB)

Propaga la siguiente observación válida hacia atrás en el tiempo ($\text{axis}=0$). Los NaN finales o persistentes se rellenan con 0.

Imputación por Interpolación Lineal

Estima los valores faltantes mediante interpolación lineal entre los puntos observados vecinos más cercanos en el tiempo ($\text{axis}=0$), extrapolando en los extremos si es posible. Los NaN restantes se rellenan con 0.

4.7.3 Métodos Basados en Modelos Predictivos (Deep Learning)

Imputación con Transformer y SAITS

Se utilizan las arquitecturas Transformer y SAITS implementadas en la biblioteca PyPOTS [?], cuyas funcionalidades detalladas se pueden consultar en su documentación¹. El proceso se divide en:

1. *Entrenamiento*: Una función dentro de `imputation_methods` instancia el modelo PyPOTS seleccionado (SAITS o Transformer) con hiperparámetros específicos. El modelo se entrena sobre los conjuntos de entrenamiento y validación ($\mathcal{X}_{\text{missing}}^{\text{train/val}}, \mathbf{M}^{\text{train/val}}$), utilizando un optimizador estándar (Adam), un planificador de tasa de aprendizaje (ReduceLROnPlateau) y una función de pérdida apropiada (SmoothL1Loss). El resultado es un modelo con parámetros θ^* optimizados.

¹ <https://docs.pypots.com/en/latest/pypots.imputation.html>

2. *Imputación (Inferencia)*: Otra función del módulo toma el modelo entrenado f_{θ^*} y lo aplica al conjunto de test $(\mathcal{X}_{\text{missing}}^{\text{test}}, \mathbf{M}^{\text{test}})$ para generar la predicción completada $\hat{\mathcal{X}}^{\text{test}}$.

4.8 EVALUACIÓN DEL RENDIMIENTO

El rendimiento final de cada método se evalúa utilizando el módulo `pampaneira_imputation.evaluation`. Este calcula las métricas MAE, MSE, RMSE y MRE comparando las predicciones $\hat{\mathcal{X}}^{\text{test}}$ con el ground truth $\mathcal{X}_{\text{orig}}^{\text{test}}$ únicamente en las posiciones indicadas por la máscara artificial $\mathbf{M}_{\text{artificial}}^{\text{test}}$.

CONCLUSIONES Y TRABAJO FUTURO

5.1 CONCLUSIONES

5.1.1 *Principales Contribuciones del Trabajo*

5.1.2 *Impacto del Uso de Datos Sintéticos en IoT*

5.2 TRABAJO FUTURO

5.2.1 *Mejora de los Modelos Transformer*

5.2.2 *Extensiones a Otros Sistemas IoT*

5.2.3 *Posibles Aplicaciones en Diferentes Dominios*

ANEXOS

ANEXO A: CRONOGRAMA

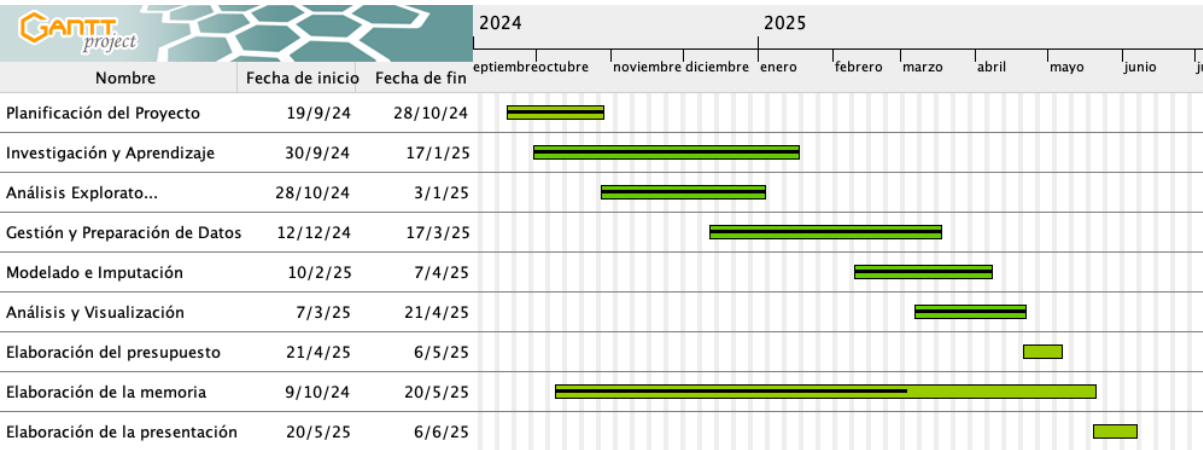


Figura 9: Diagrama de Gantt estimado para la realización del TFG.

ANEXO B: PRESUPUESTO ESTIMADO

Cuadro 1: Estimación del coste del proyecto

Concepto	Tiempo <i>(horas)</i>	Precio <i>(euros/hora)</i>	Coste <i>(euros)</i>
Parte general			
Gestión y Comunicaciones	1.4	9	12.60
Parte teórica			
Investigación y Aprendizaje	278.8	9	2509.20
Parte práctica			
Pipeline y Datos	43.6	9	392.40
Modelado y Entrenamiento	40.6	9	365.40
Imputación y Nulos	230.5	9	2074.50
Análisis y Visualización	7.0	9	63.00
Metodología y Estructura	4.4	9	39.60
Equipo: Macbook Air M2	-	-	1330.00
Coste total del proyecto	606.3	-	6786.70

APPENDIX C: DOCUMENTACIÓN DEL PAQUETE DE PYTHON

pampaneira_imputation

Release 0.1.0

Ricardo Ruiz

Apr 07, 2025

CONTENTS:

1	pampaneira_imputation package	3
1.1	Submodules	3
1.2	pampaneira_imputation.config module	3
1.3	pampaneira_imputation.data_loader module	3
1.4	pampaneira_imputation.data_preprocessor module	5
1.5	pampaneira_imputation.evaluation module	9
1.6	pampaneira_imputation.imputation_methods module	9
1.7	pampaneira_imputation.utils module	11
1.8	Module contents	12
2	tests package	13
2.1	Submodules	13
2.2	tests.conftest module	13
2.3	tests.minimal_impute_function module	13
2.4	tests.minimal_test module	13
2.5	tests.test_data_preprocessor module	13
2.6	tests.test_evaluation module	13
2.7	tests.test_function_standalone module	13
2.8	tests.test_imputation_methods module	13
2.9	tests.test_utils module	13
2.10	Module contents	13
	Python Module Index	15
	Index	17

Add your content using `reStructuredText` syntax. See the [reStructuredText](#) documentation for details.

PAMPANEIRA_IMPUTATION PACKAGE

1.1 Submodules

1.2 pampaneira_imputation.config module

1.3 pampaneira_imputation.data_loader module

```
pampaneira_imputation.data_loader.load_intersection_data(filepath: str =  
                                                         '../data/trafico_contamina_intersección.csv',  
                                                         date_col_original: str = 'Date',  
                                                         date_col_target: str = 'date',  
                                                         truck_pos_col: str = 'truck_pos',  
                                                         target_truck_pos: str = 'PAM_2',  
                                                         timezone: str = 'UTC') → DataFrame
```

Carga datos de intersección, filtra por posición de camión, convierte la fecha.

Args:

filepath (str, optional): Ruta al archivo CSV de intersección (por defecto: config.INTERSECTION_FILE).
date_col_original (str, optional): Nombre original de la columna de fecha en el CSV (por defecto: "Date").
date_col_target (str, optional): Nombre objetivo de la columna de fecha (por defecto: config.DATE_COL).
truck_pos_col (str, optional): Nombre de la columna de posición del camión (por defecto: config.TRUCK_POS_COL).
target_truck_pos (str, optional): Posición objetivo del camión para filtrar (por defecto: config.TARGET_TRUCK_POS).
timezone (str, optional): Zona horaria para las fechas (por defecto: config.TIMEZONE).

Returns:

pd.DataFrame: DataFrame con datos de intersección cargados, filtrados y preprocesados.

Raises:

FileNotFoundError: Si el archivo especificado no se encuentra. KeyError: Si una columna especificada no se encuentra o falla el cambio de nombre en el archivo.


```
pampaneira_imputation.data_loader.load_traffic_data(filepath: str = '../data/trafico_feb22_ago23.csv',
columns_to_use: List[str] =
['vehicles_PAM_1_OUT',
'vehicles_PAM_1_OUT_Zona_Granada', 'vehic-
cles_PAM_1_OUT_Zona_Catalunia_y_Otras',
'vehi-
cles_PAM_1_OUT_Zona_Andalucia_no_GR',
'vehicles_PAM_1_OUT_Extranjero', 'vehic-
cles_PAM_1_OUT_Zona_Comunidad_de_Madrid',
'vehi-
cles_PAM_1_OUT_Zona_Extremadura_y_Otras',
'vehicles_PAM_1_OUT_Zona_Otras',
'vehicles_PAM_1_OUT_5_seats',
'vehicles_PAM_1_OUT_+5_seats',
'vehicles_PAM_1_OUT_-5_seats',
'vehicles_PAM_1_OUT_nan_seats',
'vehicles_PAM_1_OUT_101-200_CO2',
'vehicles_PAM_1_OUT_0-100_CO2',
'vehicles_PAM_1_OUT_nan_CO2',
'vehicles_PAM_1_OUT_201-300_CO2',
'vehicles_PAM_1_OUT_+300_CO2',
'vehicles_PAM_1_IN',
'vehicles_PAM_1_IN_Zona_Granada',
'vehicles_PAM_1_IN_Zona_Catalunia_y_Otras',
'vehicles_PAM_1_IN_Zona_Andalucia_no_GR',
'vehicles_PAM_1_IN_Extranjero', 'vehic-
cles_PAM_1_IN_Zona_Comunidad_de_Madrid',
'vehi-
cles_PAM_1_IN_Zona_Extremadura_y_Otras',
'vehicles_PAM_1_IN_Zona_Otras',
'vehicles_PAM_1_IN_5_seats',
'vehicles_PAM_1_IN_+5_seats',
'vehicles_PAM_1_IN_-5_seats',
'vehicles_PAM_1_IN_nan_seats',
'vehicles_PAM_1_IN_101-200_CO2',
'vehicles_PAM_1_IN_0-100_CO2',
'vehicles_PAM_1_IN_nan_CO2',
'vehicles_PAM_1_IN_201-300_CO2',
'vehicles_PAM_1_IN_+300_CO2',
'vehicles_PAM_2_OUT',
'vehicles_PAM_2_OUT_Zona_Granada', 'vehic-
cles_PAM_2_OUT_Zona_Catalunia_y_Otras',
'vehi-
cles_PAM_2_OUT_Zona_Andalucia_no_GR',
'vehicles_PAM_2_OUT_Extranjero', 'vehic-
cles_PAM_2_OUT_Zona_Comunidad_de_Madrid',
'vehi-
cles_PAM_2_OUT_Zona_Extremadura_y_Otras',
'vehicles_PAM_2_OUT_Zona_Otras',
'vehicles_PAM_2_OUT_5_seats',
'vehicles_PAM_2_OUT_+5_seats',
'vehicles_PAM_2_OUT_-5_seats',
'vehicles_PAM_2_OUT_nan_seats',
'vehicles_PAM_2_OUT_101-200_CO2',
'vehicles_PAM_2_OUT_0-100_CO2',
'vehicles_PAM_2_OUT_nan_CO2',
'vehicles_PAM_2_OUT_201-300_CO2',
'vehicles_PAM_2_OUT_+300_CO2',
'vehicles_PAM_2_IN',
'vehicles_PAM_2_IN_Zona_Granada',
```

Carga datos de tráfico generales, selecciona columnas relevantes, convierte la fecha.

Args:

filepath (str, optional): Ruta al archivo CSV de tráfico (por defecto: config.TRAFFIC_FILE). columns_to_use (List[str], optional): Lista de columnas de tráfico a usar (por defecto: config.PAM_BUB_TRAFFIC_COLS). date_col (str, optional): Nombre de la columna de fecha (por defecto: config.DATE_COL). timezone (str, optional): Zona horaria para las fechas (por defecto: config.TIMEZONE).

Returns:

pd.DataFrame: DataFrame con datos de tráfico cargados y preprocesados.

Raises:

FileNotFoundError: Si el archivo especificado no se encuentra. KeyError: Si una columna especificada no se encuentra en el archivo.

1.4 pampaneira_imputation.data_preprocessor module

`pampaneira_imputation.data_preprocessor.add_period1_padding(df_period1: DataFrame, start_pad: Timestamp = Timestamp('2023-01-17 00:00:00+0000', tz='UTC'), end_pad: Timestamp = Timestamp('2023-03-14 23:00:00+0000', tz='UTC'), freq: str = 'h') → DataFrame`

Añade relleno de NaNs al inicio y al final de los datos del Periodo 1.

Args:

df_period1 (pd.DataFrame): DataFrame del Periodo 1 con DatetimeIndex. start_pad (pd.Timestamp, optional): Fecha de inicio para el relleno inicial

(por defecto: config.PERIOD_1_PADDING_START).

end_pad (pd.Timestamp, optional): Fecha de fin para el relleno final

(por defecto: config.PERIOD_1_PADDING_END).

freq (str, optional): Frecuencia para el rango de fechas de relleno

(por defecto: 'h').

Returns:

pd.DataFrame: DataFrame del Periodo 1 con relleno de NaNs añadido al inicio y al final.

`pampaneira_imputation.data_preprocessor.fill_missing_timestamps(df: DataFrame, start_date: Timestamp, end_date: Timestamp, freq: str = 'h', date_col: str = 'date') → DataFrame`

Rellena las marcas de tiempo horarias faltantes en un DataFrame con NaNs.

Args:

df (pd.DataFrame): DataFrame de entrada con una columna de fecha. start_date (pd.Timestamp): Fecha de inicio del rango completo. end_date (pd.Timestamp): Fecha de fin del rango completo. freq (str, optional): Frecuencia para el rango de fechas (por defecto: 'h'). date_col (str, optional): Nombre de la columna de fecha (por defecto: config.DATE_COL).

Returns:

pd.DataFrame: DataFrame con marcas de tiempo horarias completas y NaNs para los datos faltantes.

```

pampaneira_imputation.data_preprocessor.preprocess_for_imputation(df: DataFrame, feature_cols:
    list =
    ['vehicles_PAM_1_OUT',
     'veh-
     cles_PAM_1_OUT_Zona_Granada',
     'veh-
     cles_PAM_1_OUT_Zona_Catalunia_y_Otras',
     'veh-
     cles_PAM_1_OUT_Zona_Andalucia_no_GR',
     'veh-
     cles_PAM_1_OUT_Extranjero',
     'veh-
     cles_PAM_1_OUT_Zona_Comunidad_de_Madrid',
     'veh-
     cles_PAM_1_OUT_Zona_Extremadura_y_Otras',
     'veh-
     cles_PAM_1_OUT_Zona_Otras',
     'veh-
     cles_PAM_1_OUT_5_seats',
     'veh-
     cles_PAM_1_OUT_+5_seats',
     'vehicles_PAM_1_OUT_-
     5_seats',
     'veh-
     cles_PAM_1_OUT_nan_seats',
     'vehicles_PAM_1_OUT_101-
     200_CO2',
     'vehicles_PAM_1_OUT_0-
     100_CO2',
     'veh-
     cles_PAM_1_OUT_nan_CO2',
     'vehicles_PAM_1_OUT_201-
     300_CO2',
     'veh-
     cles_PAM_1_OUT_+300_CO2',
     'vehicles_PAM_1_IN', 'veh-
     cles_PAM_1_IN_Zona_Granada',
     'veh-
     cles_PAM_1_IN_Zona_Catalunia_y_Otras',
     'veh-
     cles_PAM_1_IN_Zona_Andalucia_no_GR',
     'veh-
     cles_PAM_1_IN_Extranjero',
     'veh-
     cles_PAM_1_IN_Zona_Comunidad_de_Madrid',
     'veh-
     cles_PAM_1_IN_Zona_Extremadura_y_Otras',
     'veh-
     cles_PAM_1_IN_Zona_Otras',
     'veh-
     cles_PAM_1_IN_5_seats',
     'veh-
     cles_PAM_1_IN_+5_seats',
     'vehicles_PAM_1_IN_-
     5_seats',
     'veh-
     cles_PAM_1_IN_nan_seats', 7
     'vehicles_PAM_1_IN_101-
     200_CO2',
     'vehicles_PAM_1_IN_0-

```

1.4. pampaneira_imputation.data_preprocessor module

Prepara los datos para modelos de imputación: divide, escala, ventanas, añade datos faltantes.

Args:

df (pd.DataFrame): DataFrame con DatetimeIndex y columnas de características. **feature_cols** (list, optional): Lista de columnas a usar como características

(por defecto: config.FEATURE_COLUMNS).

train_start (str, optional): Fecha de inicio del conjunto de entrenamiento

(por defecto: config.TRAIN_START_DATE).

train_end (str, optional): Fecha de fin del conjunto de entrenamiento

(por defecto: config.TRAIN_END_DATE).

val_start (str, optional): Fecha de inicio del conjunto de validación

(por defecto: config.VAL_START_DATE).

val_end (str, optional): Fecha de fin del conjunto de validación

(por defecto: config.VAL_END_DATE).

test_start (str, optional): Fecha de inicio del conjunto de prueba

(por defecto: config.TEST_START_DATE).

test_end (str, optional): Fecha de fin del conjunto de prueba

(por defecto: config.TEST_END_DATE).

n_steps (int, optional): Tamaño de la ventana deslizante

(por defecto: config.N_STEPS).

missing_rate (float, optional): Proporción de valores a convertir en NaN

(0 para desactivar) (por defecto: config.MISSING_RATE).

missing_pattern (str, optional): Patrón de datos faltantes: 'point', 'subseq'

o 'block' (por defecto: config.MISSING_PATTERN).

****missingness_kwargs**: Argumentos adicionales para create_missingness

(ej., min/max_length).

Returns:

Dict: Un diccionario que contiene divisiones de datos procesados

(train_X, val_X, test_X), datos originales (sin máscara) (train_X_ori, etc.), el escalador, número de características, y número de pasos. También incluye máscaras indicadoras.

`pampaneira_imputation.data_preprocessor.split_by_period(df: DataFrame, period_1_start: Timestamp = Timestamp('2023-01-17 17:00:00+0000', tz='UTC'), period_1_end: Timestamp = Timestamp('2023-03-14 11:00:00+0000', tz='UTC'), period_2_start: Timestamp = Timestamp('2023-06-06 13:00:00+0000', tz='UTC'), period_2_end: Timestamp = Timestamp('2023-06-27 00:00:00+0000', tz='UTC'), date_col: str = 'date') → Tuple[DataFrame, DataFrame]`

Divide el DataFrame en dos periodos basados en fechas predefinidas.

Args:

df (pd.DataFrame): DataFrame de entrada con una columna de fecha

o DatetimeIndex.

period_1_start (pd.Timestamp, optional): Fecha de inicio del Periodo 1
(por defecto: config.PERIOD_1_START).

period_1_end (pd.Timestamp, optional): Fecha de fin del Periodo 1
(por defecto: config.PERIOD_1_END).

period_2_start (pd.Timestamp, optional): Fecha de inicio del Periodo 2
(por defecto: config.PERIOD_2_START).

period_2_end (pd.Timestamp, optional): Fecha de fin del Periodo 2
(por defecto: config.PERIOD_2_END).

date_col (str, optional): Nombre de la columna de fecha si no se usa el índice
(por defecto: config.DATE_COL).

Returns:

Tuple[pd.DataFrame, pd.DataFrame]: Tupla que contiene el DataFrame del Periodo 1 y el DataFrame del Periodo 2.

1.5 pampaneira_imputation.evaluation module

`pampaneira_imputation.evaluation.calculate_imputation_metrics`(*y_true: ndarray, y_pred: ndarray, indicating_mask: ndarray*) → Dict[str, float]

Calcula MAE, MSE, RMSE, MRE para valores imputados donde la máscara es 1.

Args:

y_true (np.ndarray): Datos verdaderos (potencialmente con NaNs donde faltaban originalmente). *y_pred* (np.ndarray): Datos imputados. *indicating_mask* (np.ndarray): Máscara donde 1 indica un valor faltante que fue imputado,

0 indica un valor observado.

Returns:

Dict[str, float]: Diccionario que contiene 'mae', 'mse', 'rmse', 'mre'.

`pampaneira_imputation.evaluation.evaluate_all_methods`(*preprocessed_data: Dict, imputed_results: Dict[str, ndarray], methods_to_evaluate: list = ['median', 'mean', 'linear', 'ffill_bfill', 'bfill_ffill', 'saits']*) → DataFrame

Evalúa múltiples métodos de imputación usando los resultados del conjunto de prueba.

Args:

preprocessed_data (Dict): Diccionario de preprocess_for_imputation. *imputed_results* (Dict[str, np.ndarray]): Diccionario que mapea nombres de métodos a arrays NumPy imputados (conjunto de prueba). *methods_to_evaluate* (list, optional): Lista de claves en *imputed_results* para evaluar.

(por defecto: ['median', 'mean', 'linear', 'ffill_bfill', 'bfill_ffill', 'saits'])

Returns:

pd.DataFrame: DataFrame de Pandas que resume MAE, MSE, RMSE, MRE para cada método.

1.6 pampaneira_imputation.imputation_methods module

`pampaneira_imputation.imputation_methods.impute_forward_backward`(*X_missing: ndarray*) → Tuple[ndarray, ndarray]

Realiza la imputación forward-fill y backward-fill en un array 3D.

Aplica forward-fill seguido de backward-fill, y backward-fill seguido de forward-fill, para imputar valores faltantes en cada muestra del array de entrada.

Args:

X_missing (np.ndarray): Array NumPy 3D de entrada con valores faltantes (NaNs).

Forma: (n_samples, n_steps, n_features).

Returns:

Tuple[np.ndarray, np.ndarray]: Tupla que contiene dos arrays NumPy 3D:

- El primero es el resultado de forward-fill seguido de backward-fill.
- El segundo es el resultado de backward-fill seguido de forward-fill.

`pampaneira_imputation.imputation_methods.impute_linear(X_missing: ndarray) → ndarray`

Realiza la interpolación lineal para imputar valores faltantes en un array 3D.

Aplica la interpolación lineal a lo largo del eje del tiempo (axis=0) para cada muestra en el array de entrada.

Args:

X_missing (np.ndarray): Array NumPy 3D de entrada con valores faltantes (NaNs).

Forma: (n_samples, n_steps, n_features).

Returns:

np.ndarray: Array NumPy 3D con valores faltantes imputados usando interpolación lineal.

`pampaneira_imputation.imputation_methods.impute_mean(X_missing: ndarray) → ndarray`

Imputa los valores faltantes en un array 3D usando la media por característica.

Calcula la media de cada característica (a través de muestras y pasos de tiempo) y reemplaza los valores NaN con la media calculada para esa característica.

Args:

X_missing (np.ndarray): Array NumPy 3D de entrada con valores faltantes (NaNs).

Forma: (n_samples, n_steps, n_features).

Returns:

np.ndarray: Array NumPy 3D con valores faltantes imputados.

`pampaneira_imputation.imputation_methods.impute_mean_sample_wise(data)`

Imputa los valores faltantes usando la media de cada muestra (a través del tiempo).

Si toda una característica para una muestra es NaN, usa un valor de reserva.

Args:

data (ndarray): Array NumPy de forma (n_samples, n_timestamps, n_features)

Datos de entrada con valores NaN para ser imputados.

Returns:

ndarray: Array NumPy de forma (n_samples, n_timestamps, n_features)

Datos con valores NaN imputados.

`pampaneira_imputation.imputation_methods.impute_median(X_missing: ndarray) → ndarray`

Imputa los valores faltantes en un array 3D usando la mediana por característica.

Calcula la mediana de cada característica (a través de muestras y pasos de tiempo) y reemplaza los valores NaN con la mediana calculada para esa característica.

Args:

X_missing (np.ndarray): Array NumPy 3D de entrada con valores faltantes (NaNs).
Forma: (n_samples, n_steps, n_features).

Returns:

np.ndarray: Array NumPy 3D con valores faltantes imputados.

pampaneira_imputation.imputation_methods.**impute_median_sample_wise**(data)

Imputa los valores faltantes usando la mediana de cada muestra (a través del tiempo).

Si toda una característica para una muestra es NaN, usa un valor de reserva.

Args:

data (ndarray): Array NumPy de forma (n_samples, n_timestamps, n_features)
Datos de entrada con valores NaN para ser imputados.

Returns:

ndarray: Array NumPy de forma (n_samples, n_timestamps, n_features)
Datos con valores NaN imputados.

1.7 pampaneira_imputation.utils module

pampaneira_imputation.utils.**create_missingness**(data: ndarray, rate: float, pattern: str, **kwargs) → ndarray

Introduce valores faltantes (NaN) en un array NumPy 3D (muestras, pasos, características).

Modifica el array in situ por eficiencia, pero también lo devuelve.

Args:

data (np.ndarray): Array NumPy 3D de entrada. Forma: (n_samples, n_steps, n_features). rate (float): Tasa de datos faltantes a introducir (entre 0 y 1). pattern (str): Patrón de datos faltantes: 'point', 'subseq' o 'block'. **kwargs: Argumentos adicionales dependientes del patrón:

- Para 'subseq': min_missing_len, max_missing_len.
- Para 'block': min_missing_len, max_missing_len.

Returns:

np.ndarray: Array NumPy 3D con valores faltantes introducidos.

pampaneira_imputation.utils.**reshape_imputed_to_df**(imputed_data: ndarray, original_index: DatetimeIndex, columns: List[str], n_steps: int) → DataFrame

Remodela los datos imputados 3D (muestras, pasos, características) de vuelta a un DataFrame 2D.

Asume que el índice original corresponde al *inicio* de cada ventana. Reconstruye la línea de tiempo completa.

Args:

imputed_data (np.ndarray): Array NumPy 3D de datos imputados.
Forma: (n_samples, n_steps, n_features).

original_index (pd.DatetimeIndex): DatetimeIndex original correspondiente al conjunto de datos a imputar.
columns (List[str]): Lista de nombres de columnas para el DataFrame reconstruido. n_steps (int): Número de pasos de tiempo en cada ventana.

Returns:

pd.DataFrame: DataFrame de Pandas reconstruido a partir de los datos imputados.

`pampaneira_imputation.utils.sliding_window(data: ndarray, n_steps: int) → ndarray`

Crea ventanas deslizantes a partir de datos secuenciales.

Args:

data (np.ndarray): Array NumPy 2D o 3D de datos secuenciales.

Si es 2D, se asume forma (n_timesteps, n_features). Si es 3D, se asume forma (n_samples, n_timesteps, n_features).

n_steps (int): Tamaño de cada ventana deslizante.

Returns:

np.ndarray: Array NumPy 3D de ventanas deslizantes.

Forma: (n_windows, n_steps, n_features) si la entrada era 2D,

o (n_samples, n_windows, n_steps, n_features) si la entrada era 3D. En este caso, siempre devuelve 3D con forma (n_windows, n_steps, n_features).

1.8 Module contents

Módulo de inicialización para el paquete `pampaneira_imputation`.

TESTS PACKAGE

2.1 Submodules

2.2 tests.conftest module

2.3 tests.minimal_impute_function module

2.4 tests.minimal_test module

2.5 tests.test_data_preprocessor module

2.6 tests.test_evaluation module

2.7 tests.test_function_standalone module

2.8 tests.test_imputation_methods module

2.9 tests.test_utils module

2.10 Module contents

PYTHON MODULE INDEX

p

- `pampaneira_imputation`, [12](#)
- `pampaneira_imputation.config`, [3](#)
- `pampaneira_imputation.data_loader`, [3](#)
- `pampaneira_imputation.data_preprocessor`, [5](#)
- `pampaneira_imputation.evaluation`, [9](#)
- `pampaneira_imputation.imputation_methods`, [9](#)
- `pampaneira_imputation.utils`, [11](#)

A

`add_period1_padding()` (in module `pampaneira_imputation.data_preprocessor`), 5

C

`calculate_imputation_metrics()` (in module `pampaneira_imputation.evaluation`), 9

`create_missingness()` (in module `pampaneira_imputation.utils`), 11

E

`evaluate_all_methods()` (in module `pampaneira_imputation.evaluation`), 9

F

`fill_missing_timestamps()` (in module `pampaneira_imputation.data_preprocessor`), 5

I

`impute_forward_backward()` (in module `pampaneira_imputation.imputation_methods`), 9

`impute_linear()` (in module `pampaneira_imputation.imputation_methods`), 10

`impute_mean()` (in module `pampaneira_imputation.imputation_methods`), 10

`impute_mean_sample_wise()` (in module `pampaneira_imputation.imputation_methods`), 10

`impute_median()` (in module `pampaneira_imputation.imputation_methods`), 10

`impute_median_sample_wise()` (in module `pampaneira_imputation.imputation_methods`), 11

L

`load_intersection_data()` (in module `pampaneira_imputation.data_loader`), 3

`load_traffic_data()` (in module `pampaneira_imputation.data_loader`), 3

M

module

`pampaneira_imputation`, 12

`pampaneira_imputation.config`, 3

`pampaneira_imputation.data_loader`, 3

`pampaneira_imputation.data_preprocessor`, 5

`pampaneira_imputation.evaluation`, 9

`pampaneira_imputation.imputation_methods`, 9

`pampaneira_imputation.utils`, 11

P

`pampaneira_imputation`
module, 12

`pampaneira_imputation.config`
module, 3

`pampaneira_imputation.data_loader`
module, 3

`pampaneira_imputation.data_preprocessor`
module, 5

`pampaneira_imputation.evaluation`
module, 9

`pampaneira_imputation.imputation_methods`
module, 9

`pampaneira_imputation.utils`
module, 11

`preprocess_for_imputation()` (in module `pampaneira_imputation.data_preprocessor`), 6

R

`reshape_imputed_to_df()` (in module `pampaneira_imputation.utils`), 11

S

`sliding_window()` (in module `pampaneira_imputation.utils`), 11

`split_by_period()` (in module *pam-*
paneira_imputation.data_preprocessor),
8